# Advanced Analysis of Algorithms CC5031

## ☺ Divide and Conquer Part 1☹

## Lectures 5-6

**Dr. Salman Masih**

Assistant Professor

Department of Computer Science

University of Management and Technology, Sialkot, Pakistan

# Outline

1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search







a problem to be solved

**Divide**: Break into non-overlapping subproblems of the same type



**Divide**: Break into non-overlapping subproblems of the same type



**Divide**: Break into non-overlapping subproblems of the same type

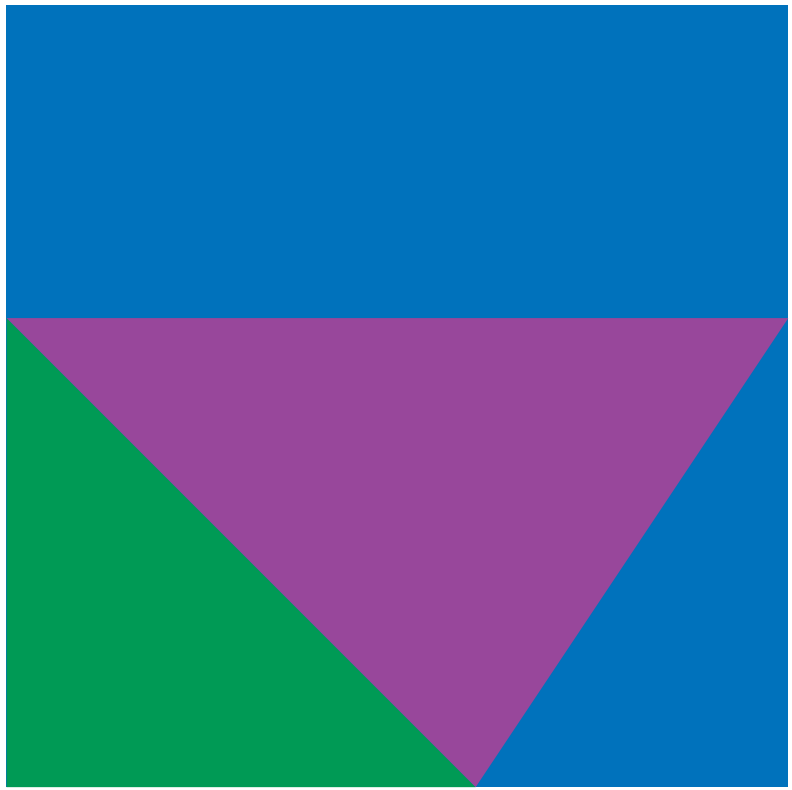


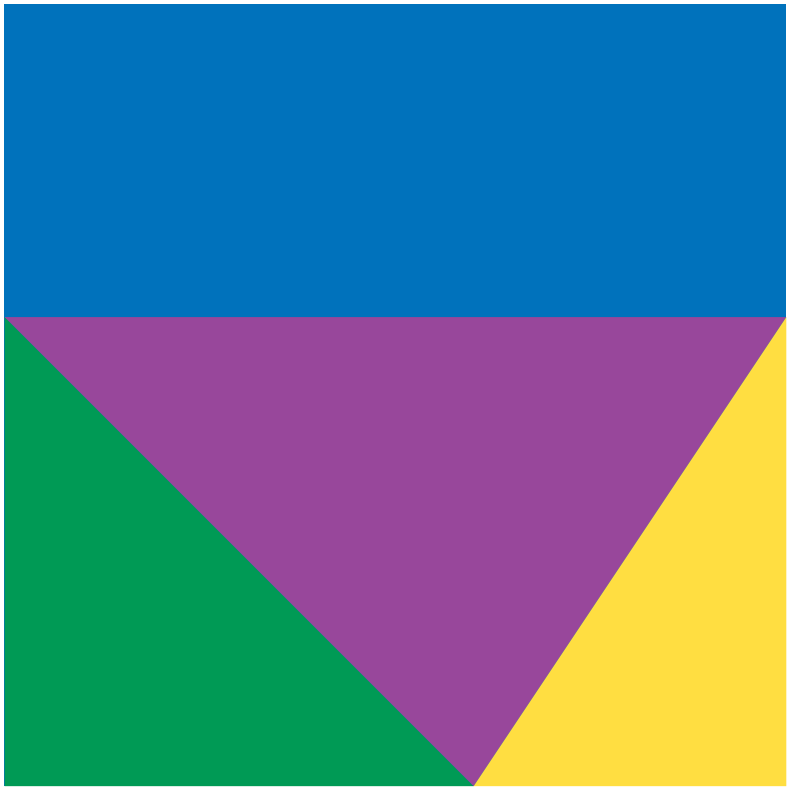
**Divide**: Break into non-overlapping subproblems of the same type

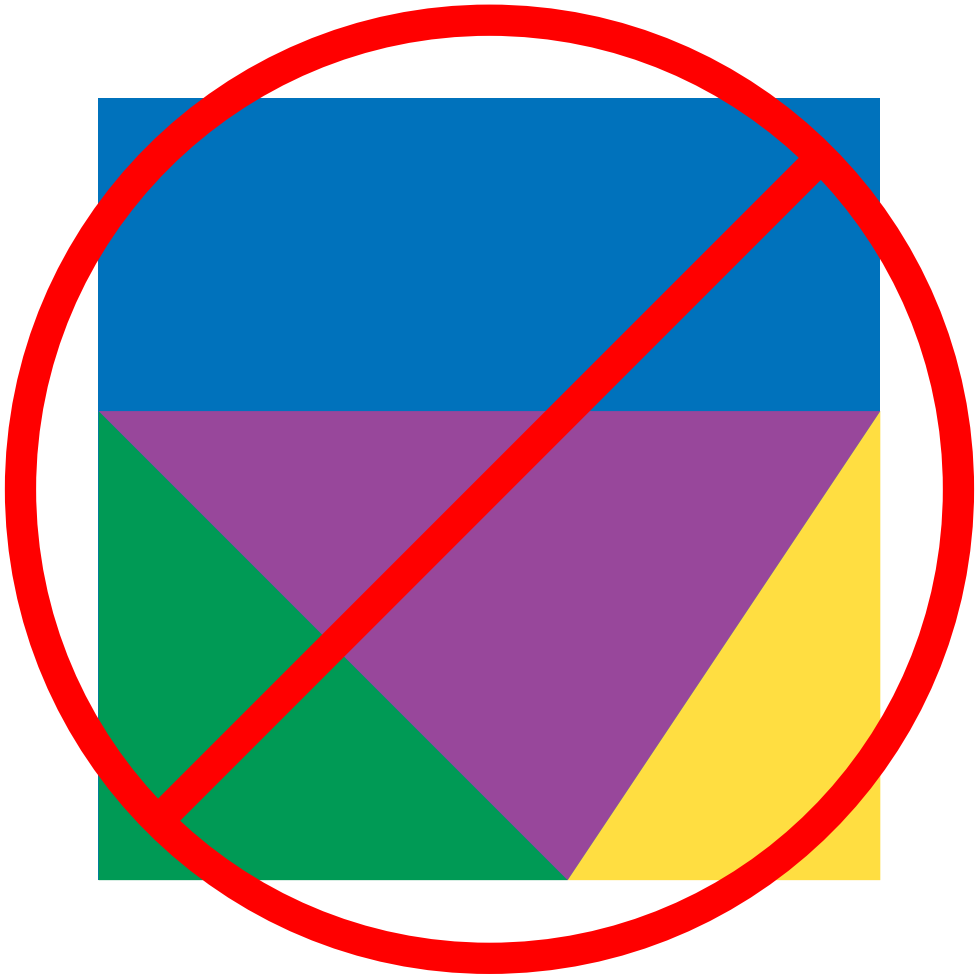












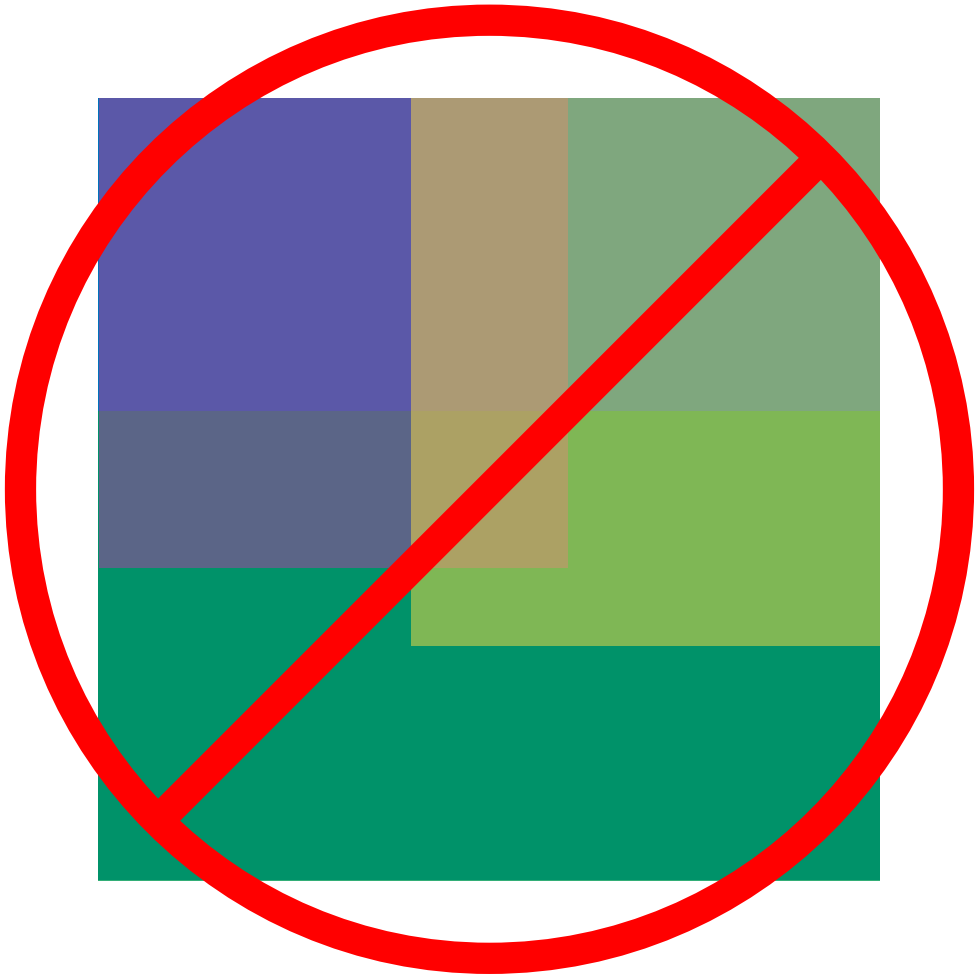
not the
same type











overlapping

**Divide**: break apart



**Divide**: break apart



# **Conquer**: solve subproblems



# **Conquer**: solve subproblems



# **Conquer**: solve subproblems



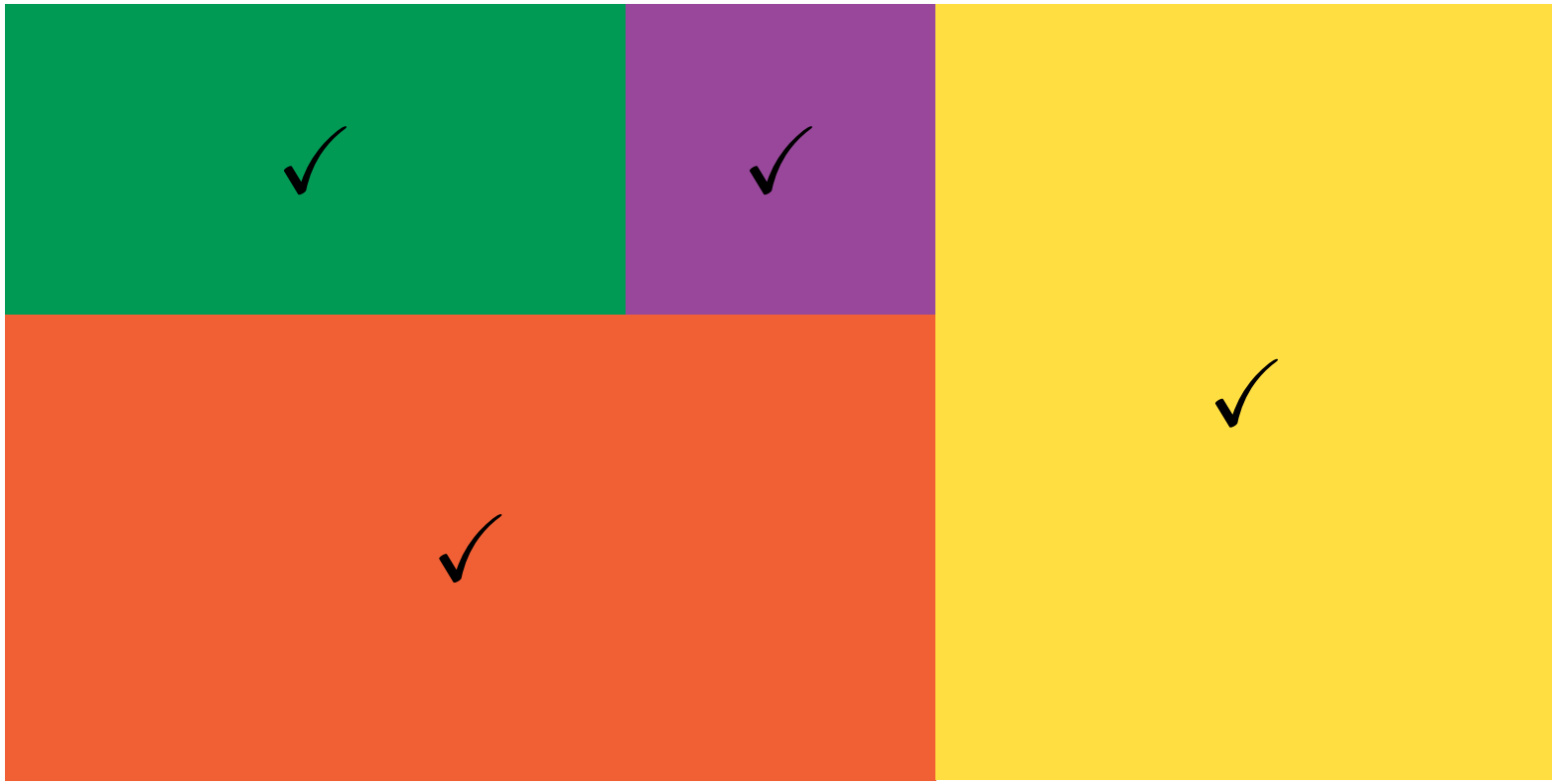
# **Conquer**: solve subproblems



# **Conquer**: solve subproblems



# **Conquer**: combine



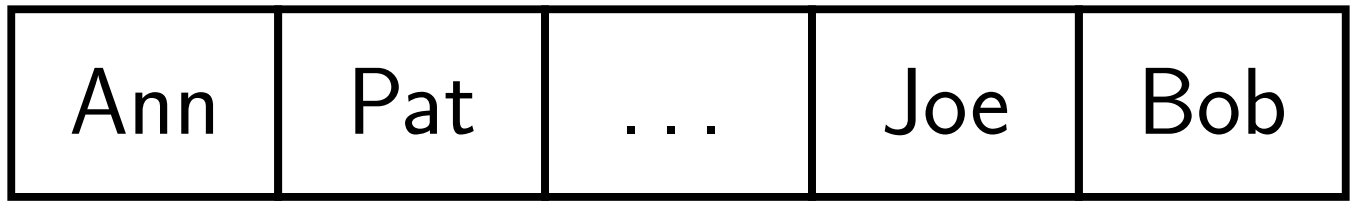


1. Break into non-overlapping subproblems of the same type
2. Solve subproblems
3. Combine results

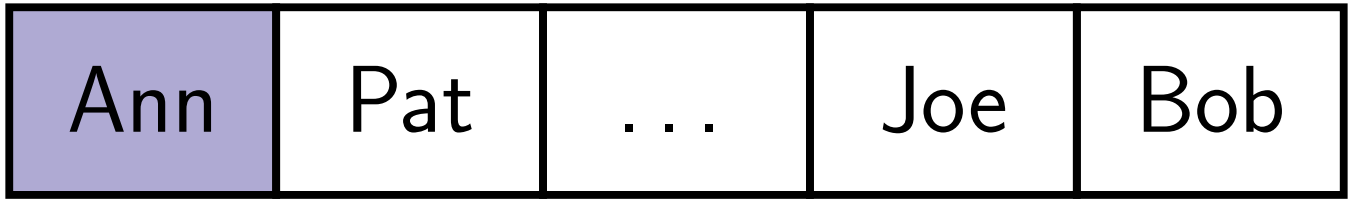
# Outline


1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search

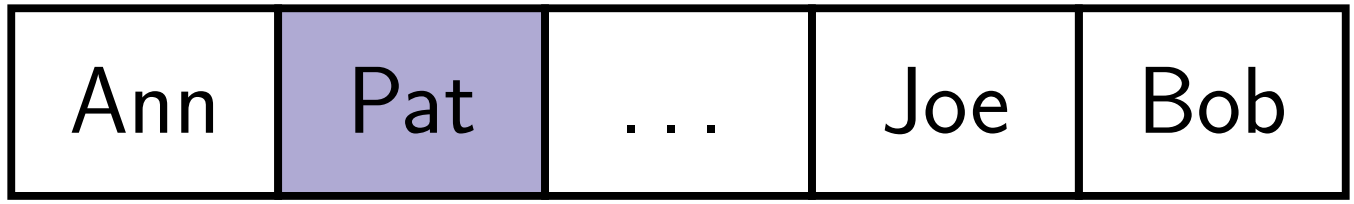
# Linear Search in Array



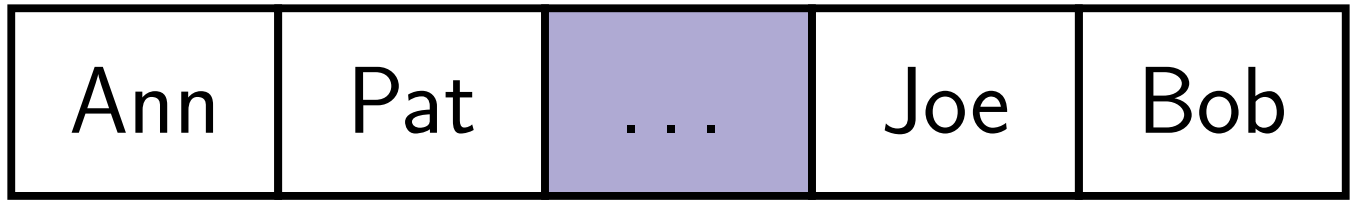
# Linear Search in Array



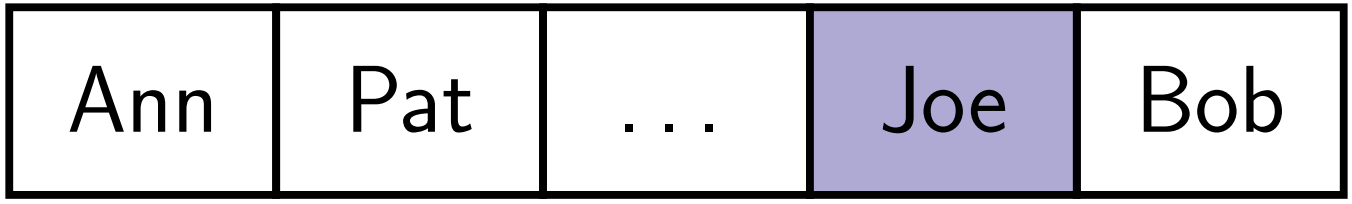
# Linear Search in Array



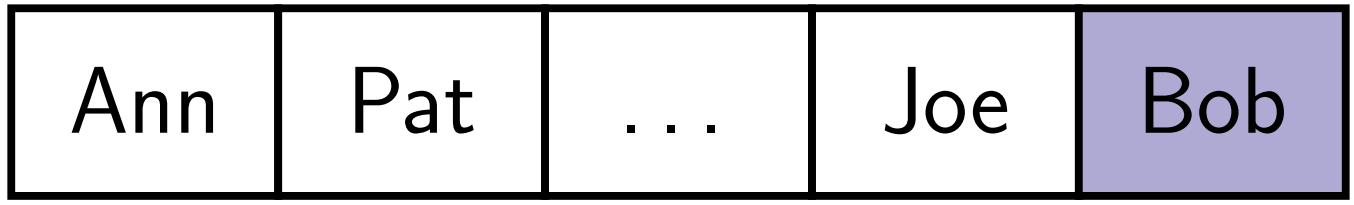
# Linear Search in Array



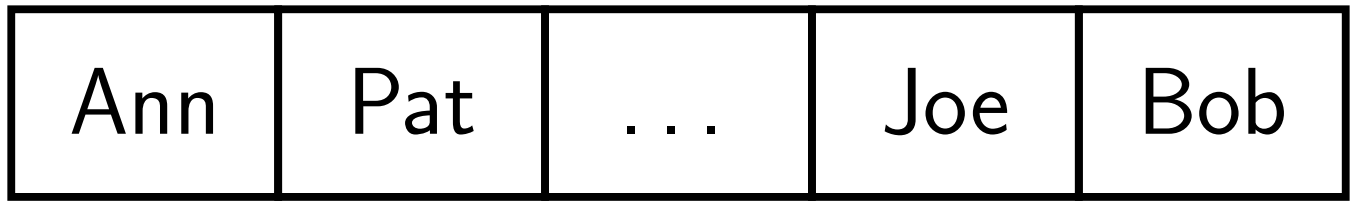
# Linear Search in Array



# Linear Search in Array



# Linear Search in Array



# Real-life Example

| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| car | voiture | auto | Auto | auto |
| table | table | tavola | Tabelle | mesa |

# Searching in an array

**Input:** An array $A$ with $n$ elements.
A key $k$.

**Output:** An index, $i$, where $A[i] = k$.
If there is no such $i$, then
NOT_FOUND.

# Recursive Solution

`LinearSearch`($A$, $low$, $high$, $key$)

# Recursive Solution

## LinearSearch($A$, $low$, $high$, $key$)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
```

# Recursive Solution

## LinearSearch($A$, $low$, $high$, $key$)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
return LinearSearch(A, low + 1, high, key)
```

# Recursive Solution

## LinearSearch($A$, $low$, $high$, $key$)

```
if high < low:
   return NOT_FOUND
if A[low] = key:
   return low
return LinearSearch(A, low + 1, high, key)
```

## LinearSearch($A$, $low$, $high$, $key$)

```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

## LinearSearch($A$, $low$, $high$, $key$)

```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

Recurrence defining worst-case time:

$$T(n) = T(n-1) + c$$

## LinearSearch($A$, $low$, $high$, $key$)

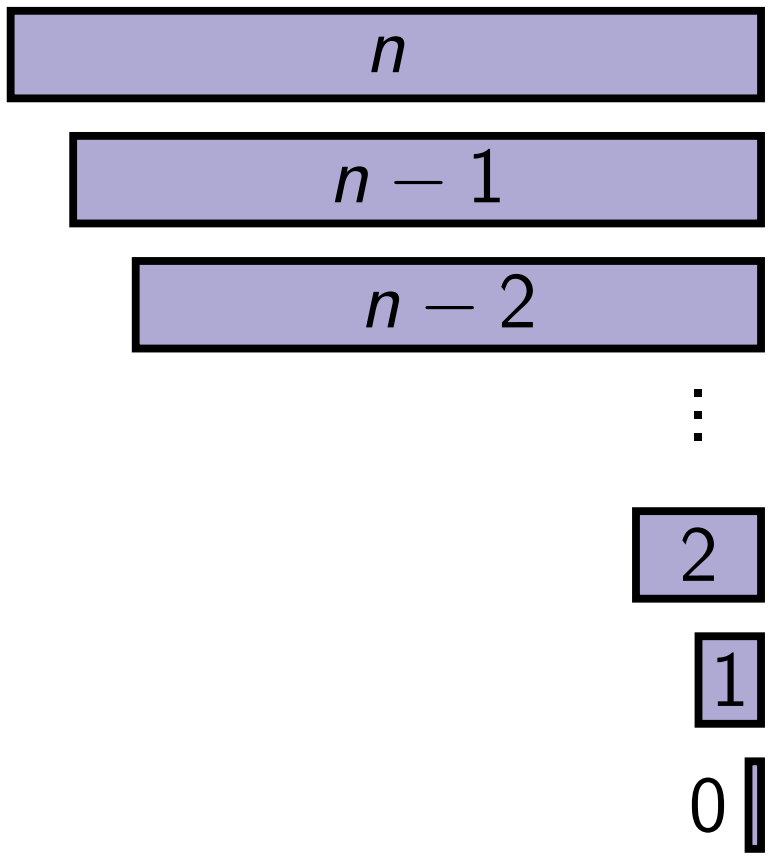
```
if high < low:
  return NOT_FOUND
if A[low] = key:
  return low
return LinearSearch(A, low + 1, high, key)
```

Recurrence defining worst-case time:

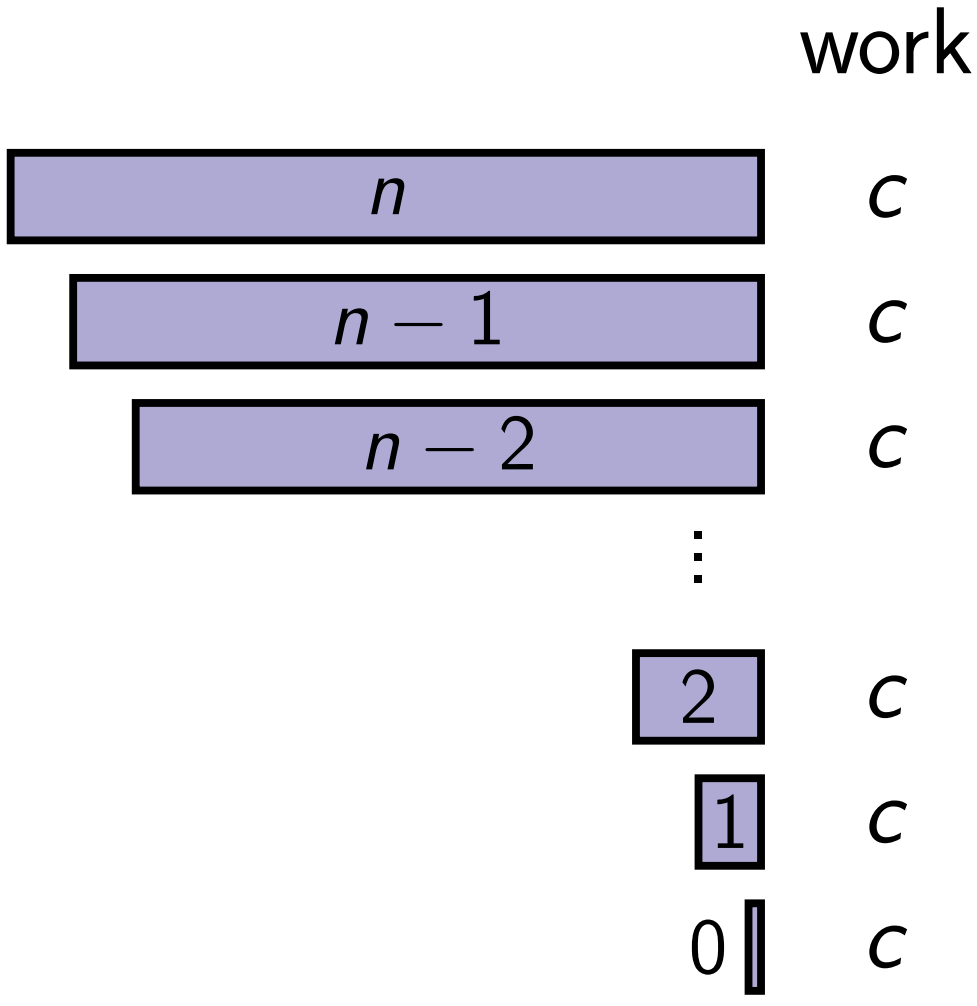
$$T(n) = T(n-1) + c$$

$$T(0) = c$$

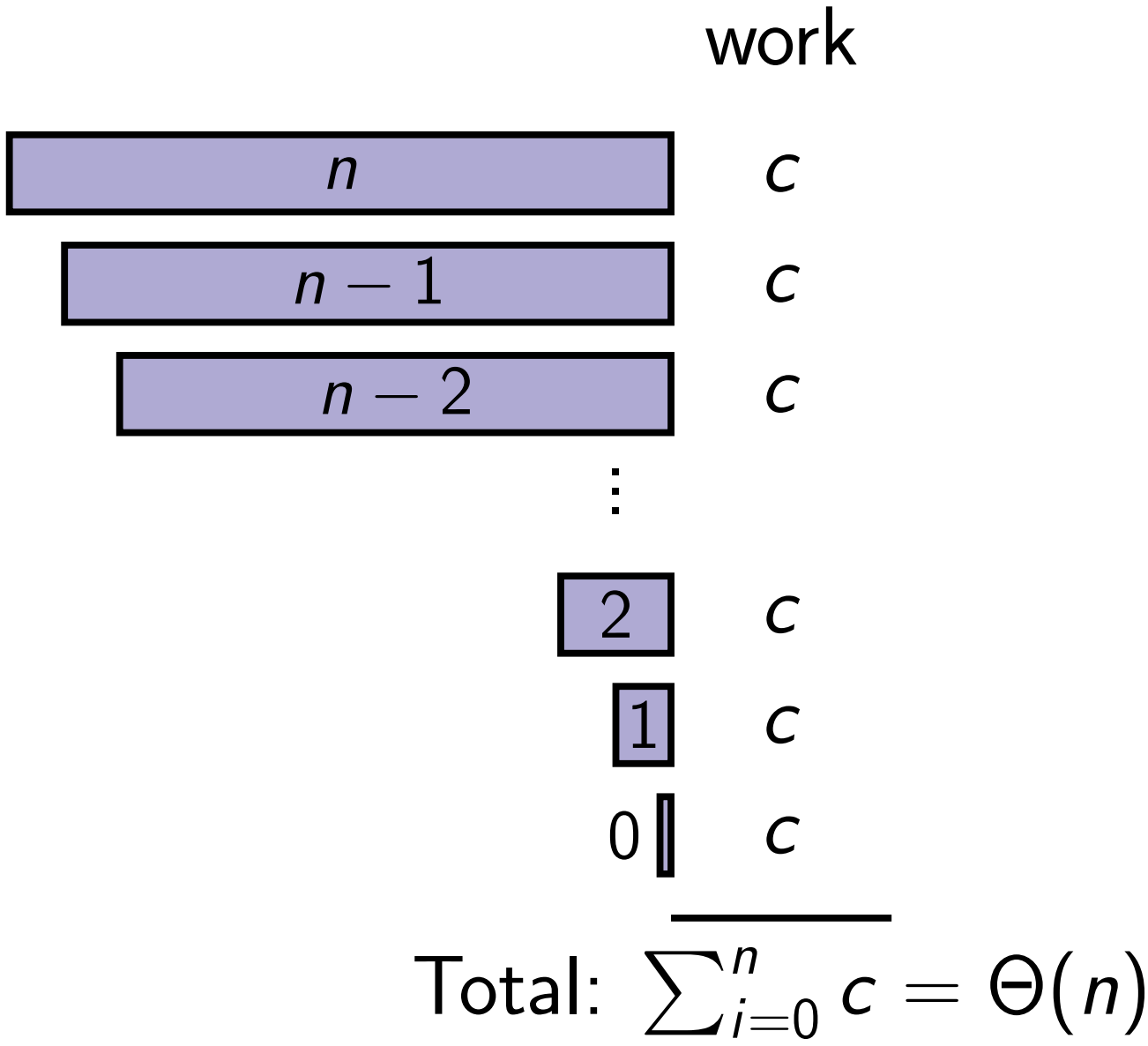
# Runtime of Linear Search



# Runtime of Linear Search



# Runtime of Linear Search



# Iterative Version

`LinearSearchIt`($A$, *low*, *high*, *key*)

```
for i from low to high:
    if A[i] = key:
        return i
return NOT_FOUND
```

# Summary

- Create a recursive solution

# Summary

- Create a recursive solution
- Define a corresponding recurrence relation, $T$

# Summary

- Create a recursive solution
- Define a corresponding recurrence relation, $T$
- Determine $T(n)$: worst-case runtime

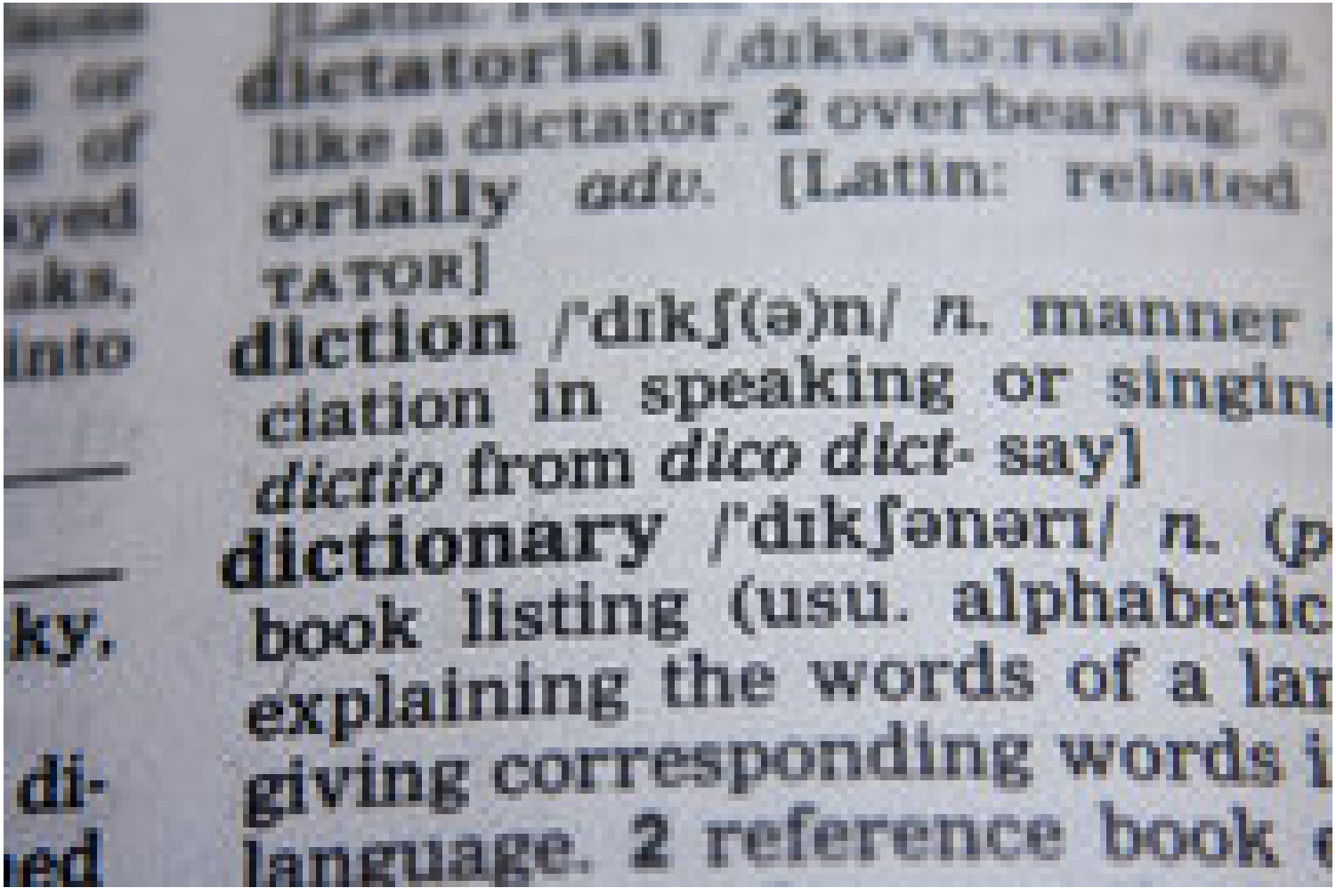
# Summary

- Create a recursive solution
- Define a corresponding recurrence relation, $T$
- Determine $T(n)$: worst-case runtime
- Optionally, create iterative solution

# Outline


1. Main Idea of Divide-and-Conquer
2. Linear Search
3. Binary Search

# Searching Sorted Data



# `BinarySearch`($A$, $low$, $high$, $key$)

# BinarySearch($A$, *low*, *high*, *key*)

```
if high < low:
   return low − 1
```

# BinarySearch($A$, $low$, $high$, $key$)

if $high < low$:
  return $low - 1$
$mid \leftarrow \left\lfloor low + \frac{high - low}{2} \right\rfloor$

# BinarySearch($A$, $low$, $high$, $key$)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
```

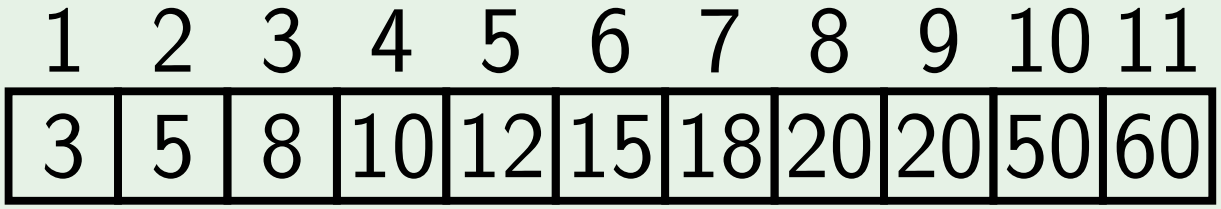
# BinarySearch($A$, $low$, $high$, $key$)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
```

# BinarySearch($A$, $low$, $high$, $key$)

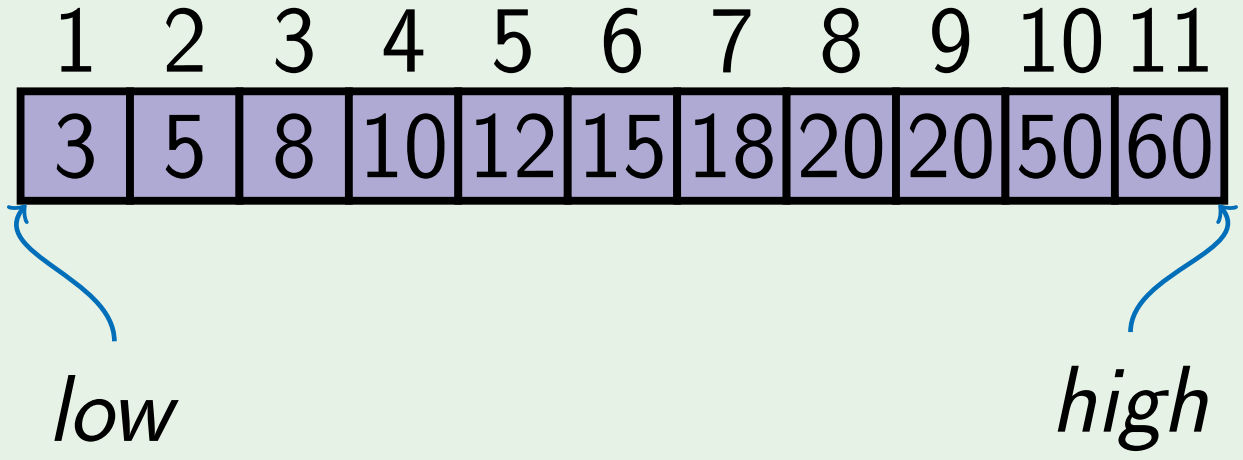
```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
else:
    return BinarySearch(A, mid + 1, high, key)
```

# Example: Searching for the key 50



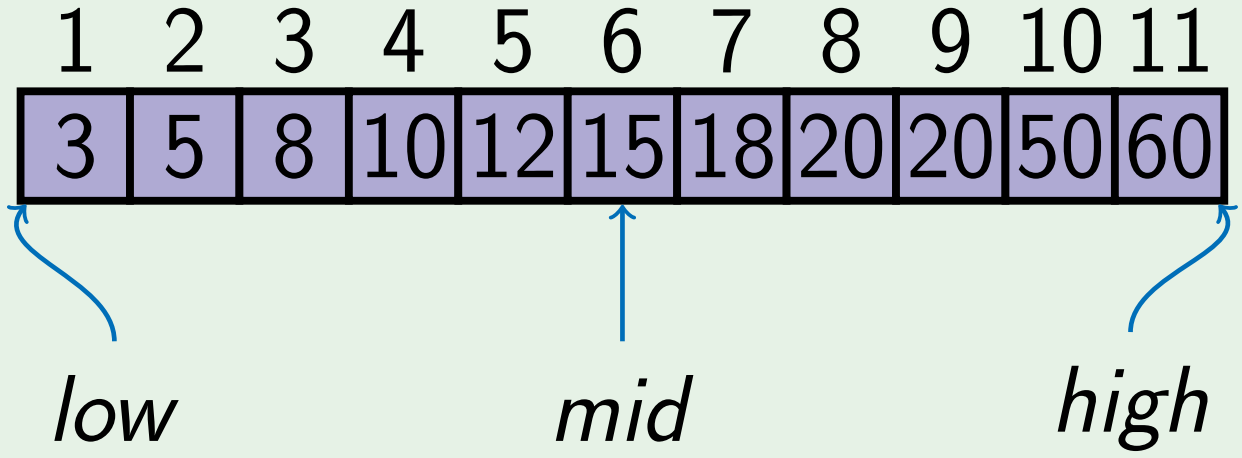
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$



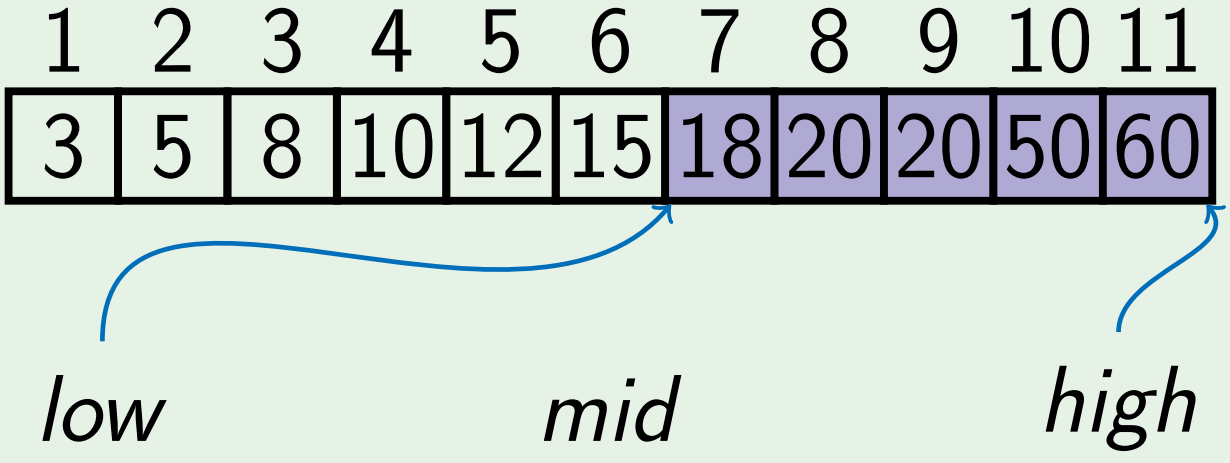
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$



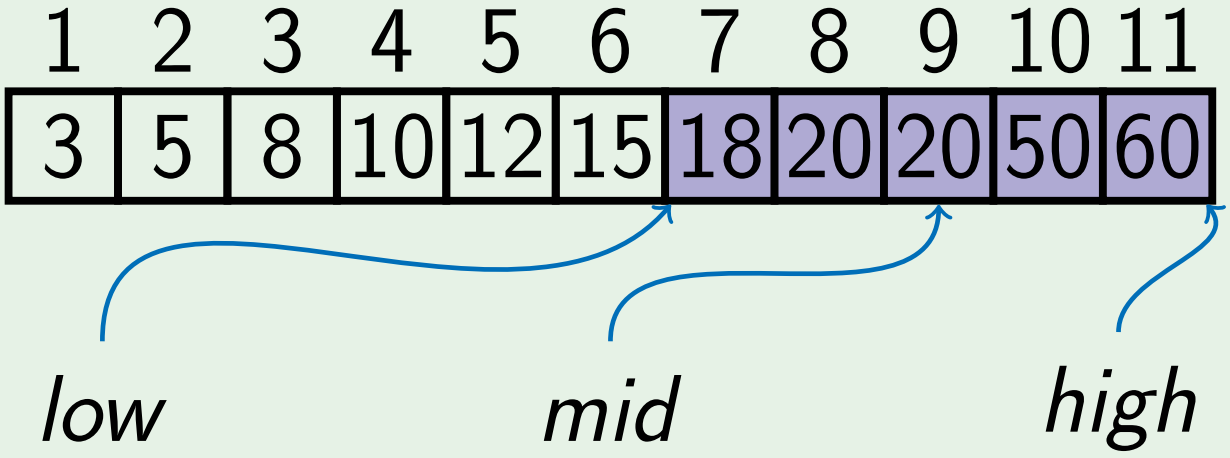
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$



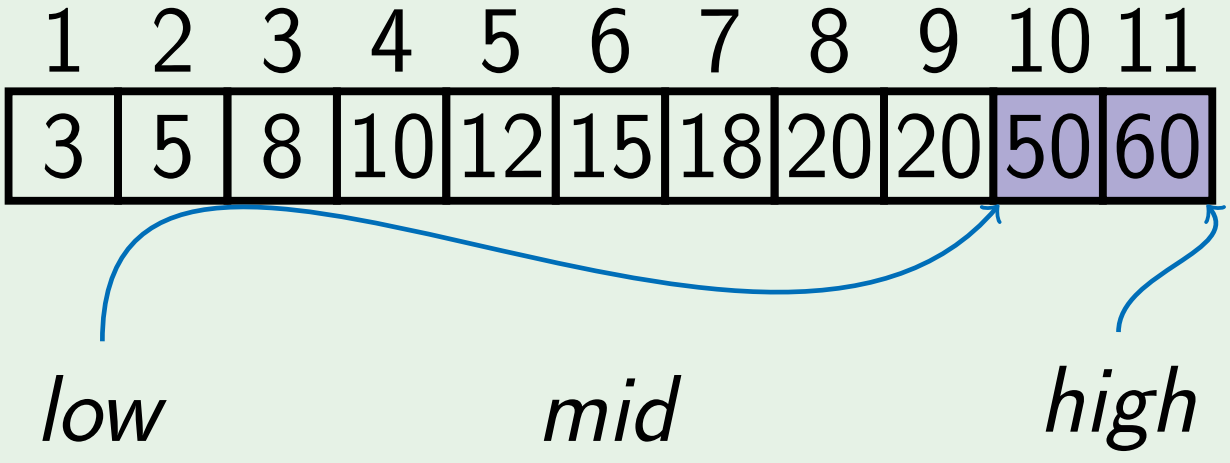
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$



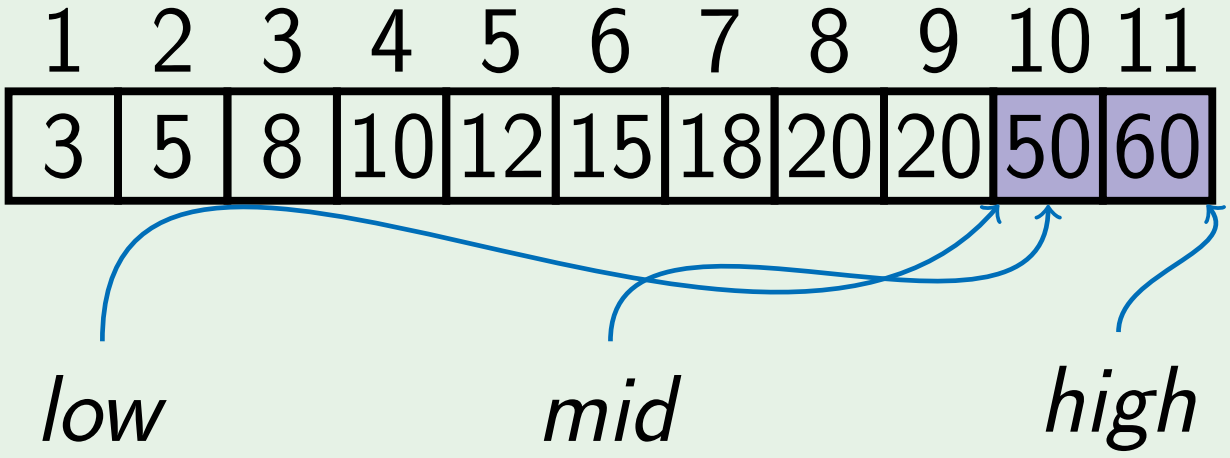
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50)$



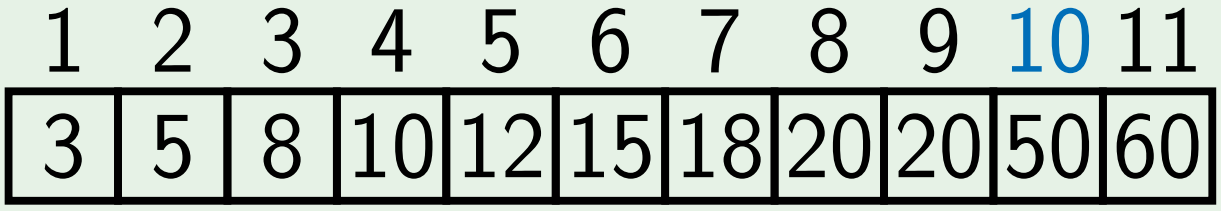
# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50)$



# Example: Searching for the key 50

$\texttt{BinarySearch}(A, 1, 11, 50)$
$\texttt{BinarySearch}(A, 7, 11, 50)$
$\texttt{BinarySearch}(A, 10, 11, 50) \rightarrow 10$



# Summary

# Summary

- Break problem into non-overlapping subproblems of the same type.

# Summary

- Break problem into non-overlapping subproblems of the same type.
- Recursively solve those subproblems.

# Summary

- Break problem into non-overlapping subproblems of the same type.
- Recursively solve those subproblems.
- Combine results of subproblems.

# BinarySearch($A$, $low$, $high$, $key$)

```
if high < low:
    return low − 1
mid ← ⌊low + (high − low)/2⌋
if key = A[mid]:
    return mid
else if key < A[mid]:
    return BinarySearch(A, low, mid − 1, key)
else:
    return BinarySearch(A, mid + 1, high, key)
```

# Binary Search Recurrence Relation

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

# Binary Search Recurrence Relation

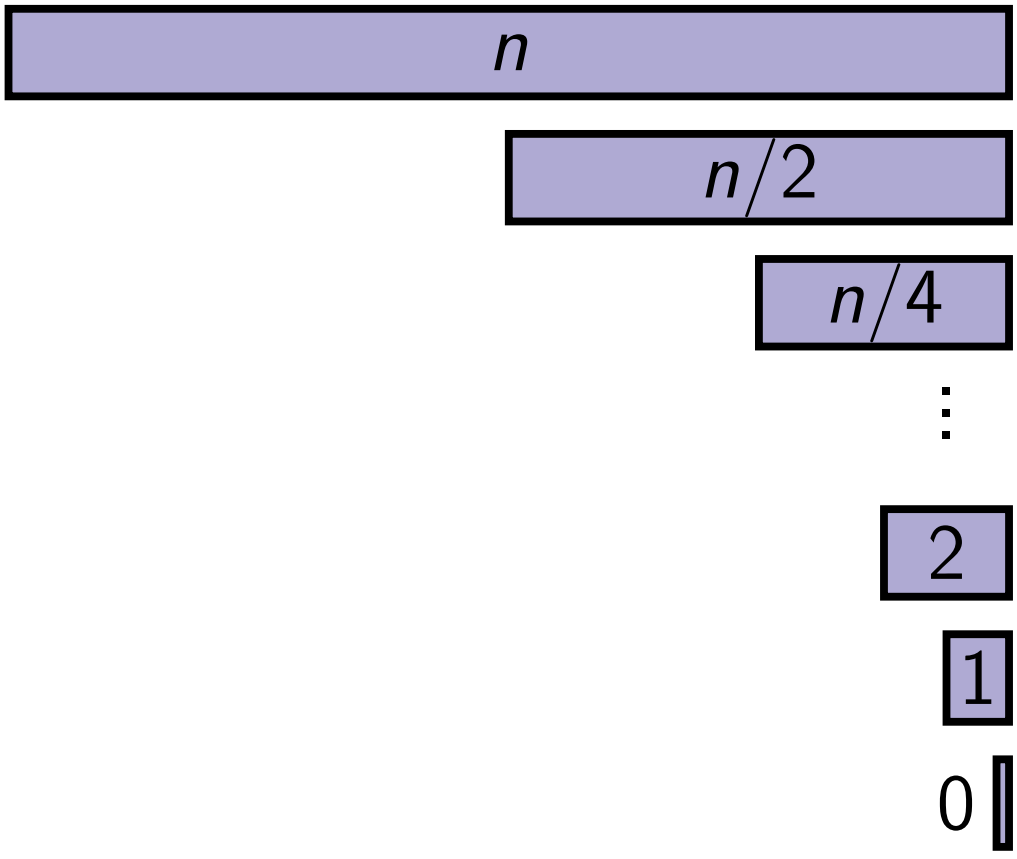
$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

# Binary Search Recurrence Relation

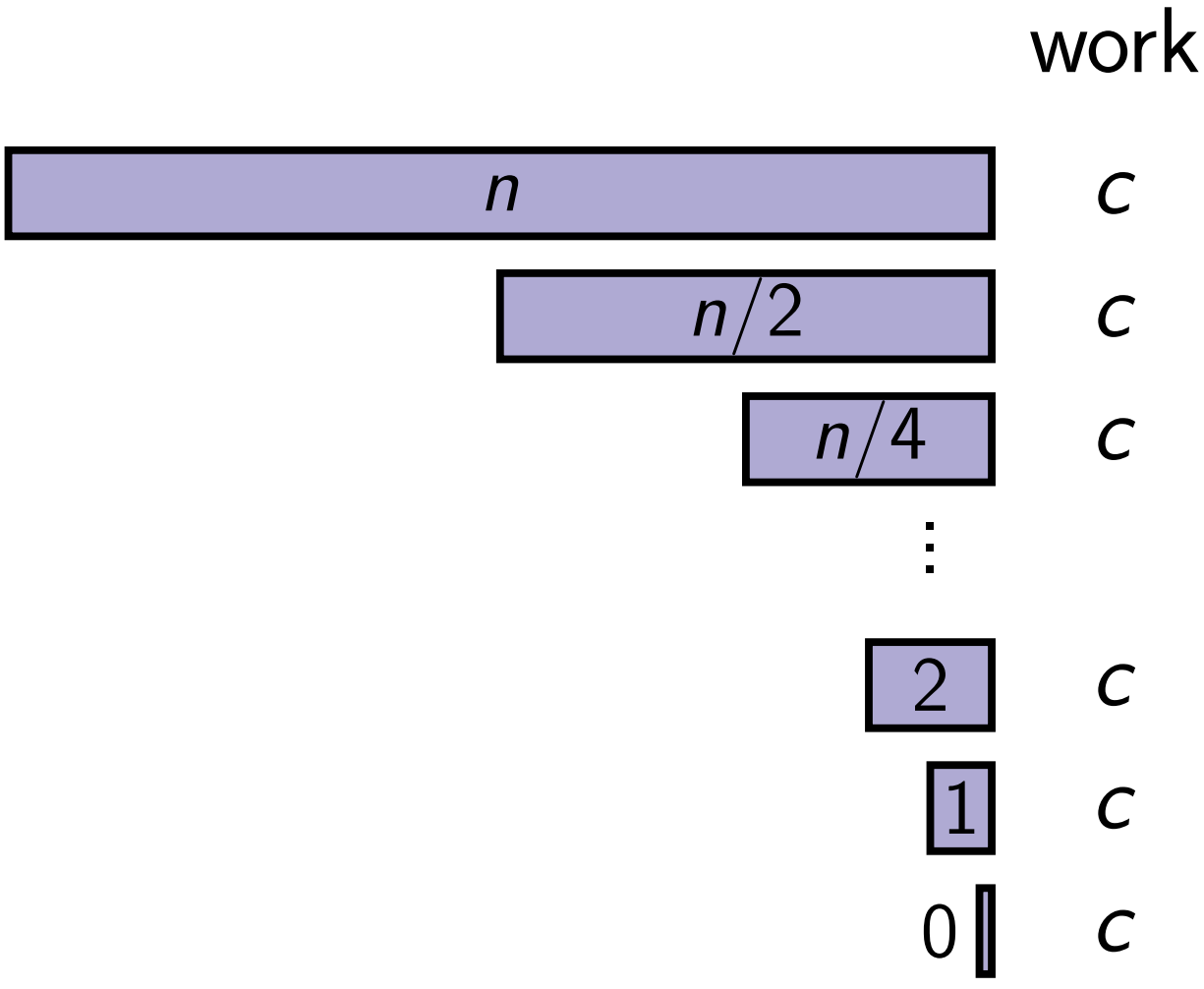
$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

$$T(0) = c$$

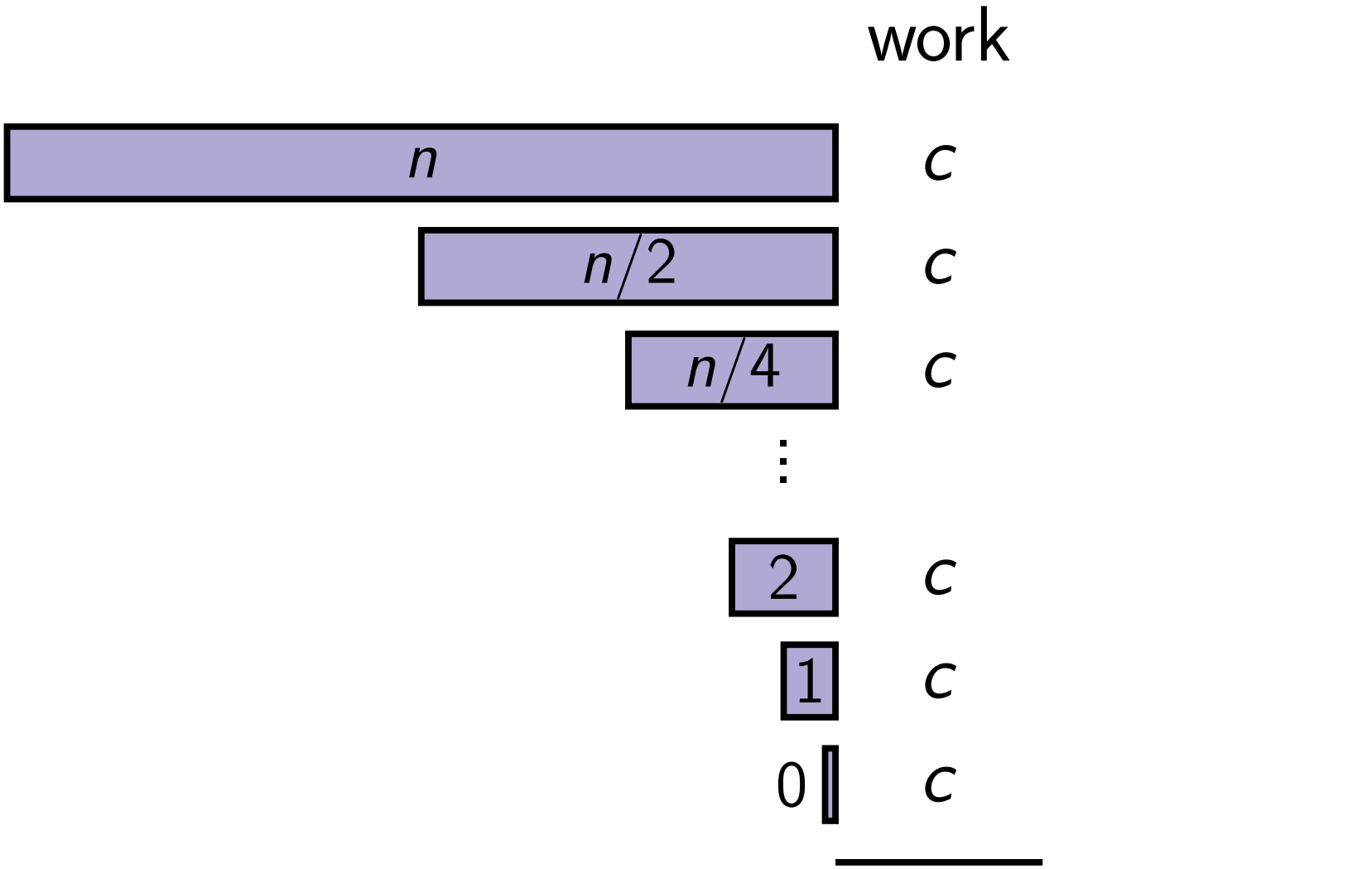
# Runtime of Binary Search



# Runtime of Binary Search



# Runtime of Binary Search




Total: $\sum_{i=0}^{\log_2 n} c = \Theta(\log_2 n)$

# Iterative Version

`BinarySearchIt`$(A, low, high, key)$

`while` $low \leq high$:

$mid \leftarrow \left\lfloor low + \frac{high - low}{2} \right\rfloor$

# Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
```

# Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
```

# Iterative Version

`BinarySearchIt`$(A, low, high, key)$

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
    else:
        low = mid + 1
```

# Iterative Version

**BinarySearchIt($A, low, high, key$)**

```
while low ≤ high:
    mid ← ⌊low + (high−low)/2⌋
    if key = A[mid]:
        return mid
    else if key < A[mid]:
        high = mid − 1
    else:
        low = mid + 1
return low − 1
```

# Real-life Example

| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| chair | chaise | sedia | Sessel | silla |
| pimple | bouton | foruncolo | Pickel | espenilla |

# Real-life Example

| **english** (sorted) | **french** (sorted) | **italian** (sorted) | **german** (sorted) | **spanish** (sorted) |
|---|---|---|---|---|
| chair | chaise | casa | Haus | casa |
| house | bouton | foruncolo | Pickel | espenilla |
| pimple | maison | sedia | Sessel | silla |

# Real-life Example

| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| chair | chaise | sedia | Sessel | silla |
| pimple | bouton | foruncolo | Pickel | espenilla |

**english**
sorted

| |
|---|
| 2 |
| 1 |
| 3 |

**spanish**
sorted

| |
|---|
| 1 |
| 3 |
| 2 |

# Real-life Example

| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| chair | chaise | sedia | Sessel | silla |
| pimple | bouton | foruncolo | Pickel | espenilla |



**english**
sorted

| |
|---|
| 2 |
| 1 |
| 3 |

**spanish**
sorted

| |
|---|
| 1 |
| 3 |
| 2 |

# Real-life Example

| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| chair | chaise | sedia | Sessel | silla |
| pimple | bouton | foruncolo | Pickel | espenilla |



**english**
sorted

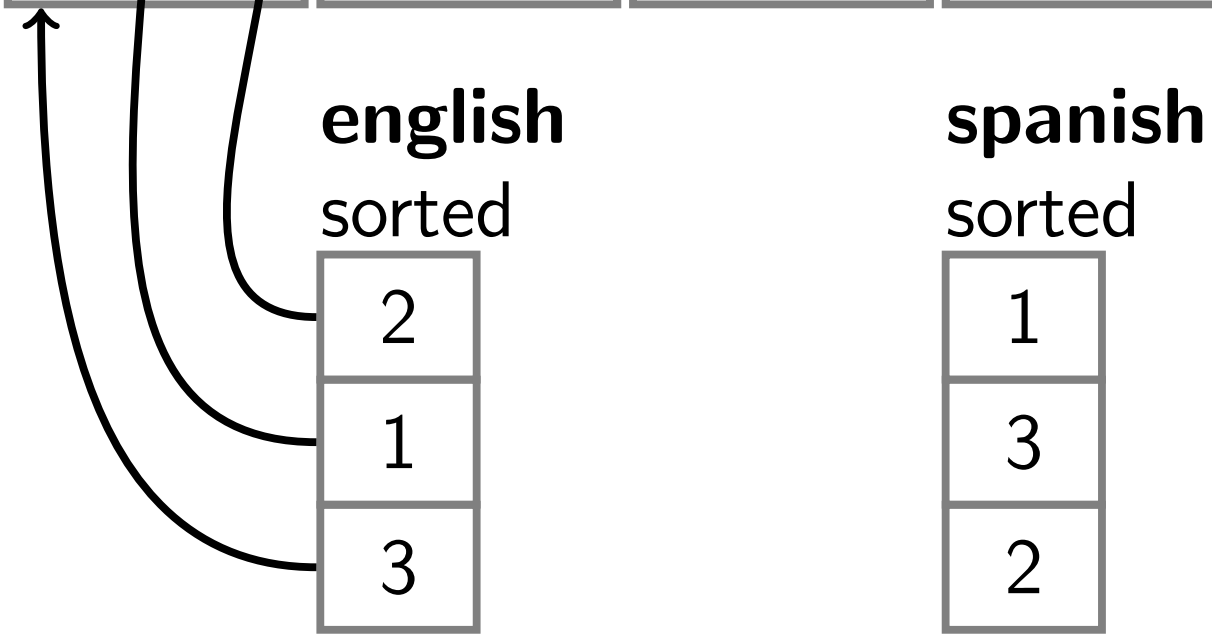
| |
|---|
| 2 |
| 1 |
| 3 |

**spanish**
sorted

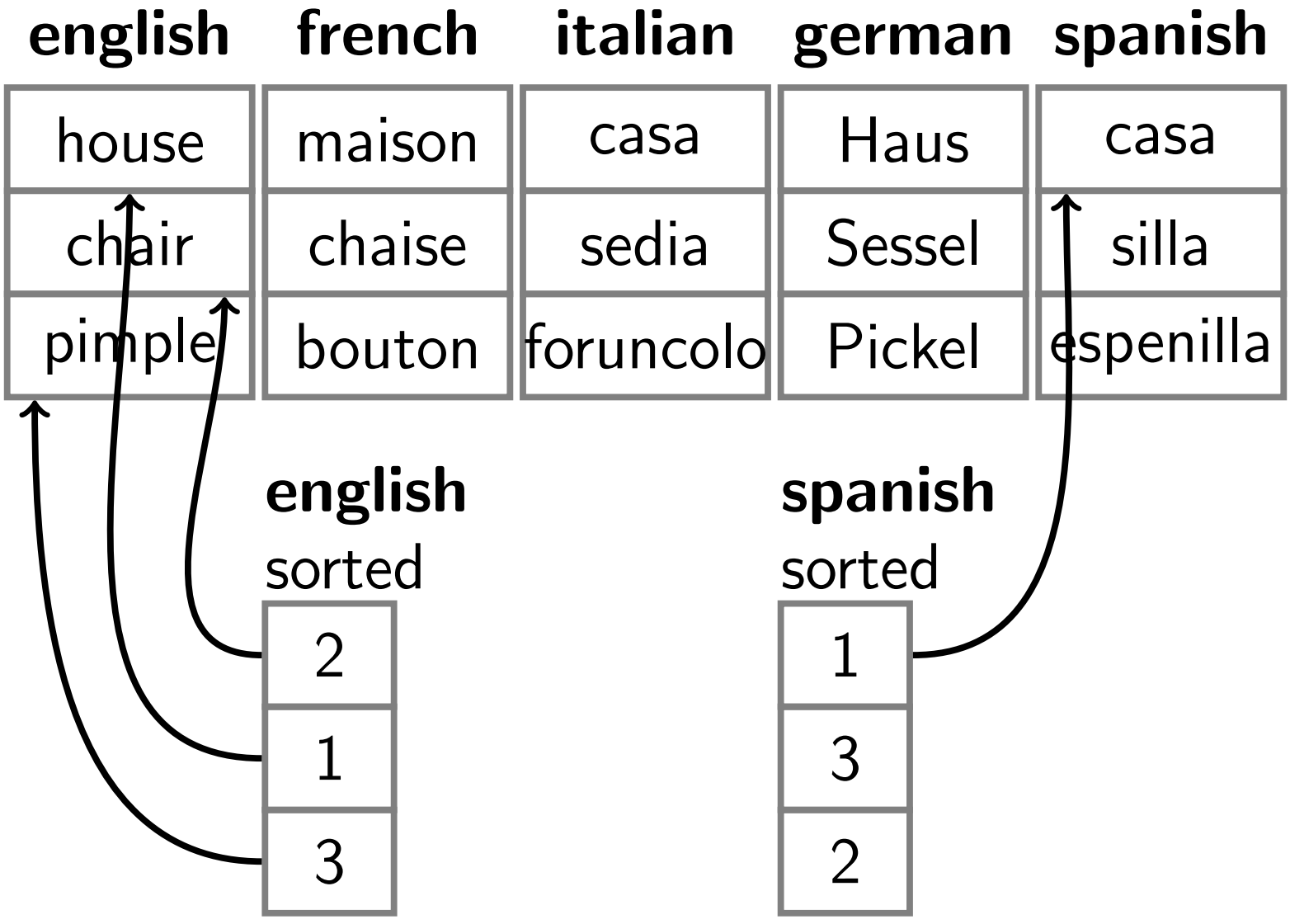
| |
|---|
| 1 |
| 3 |
| 2 |

# Real-life Example

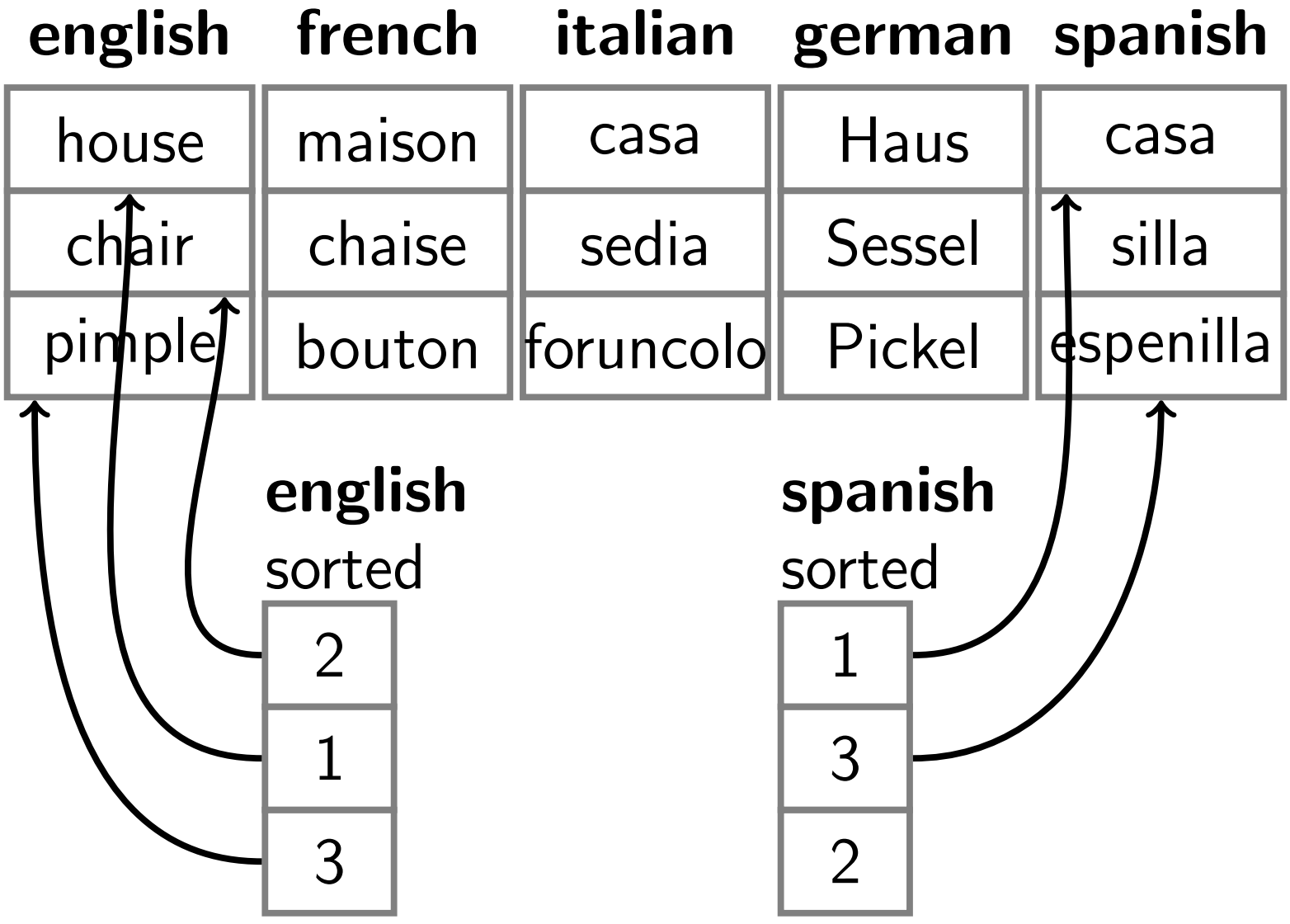
| **english** | **french** | **italian** | **german** | **spanish** |
|---|---|---|---|---|
| house | maison | casa | Haus | casa |
| chair | chaise | sedia | Sessel | silla |
| pimple | bouton | foruncolo | Pickel | espenilla |



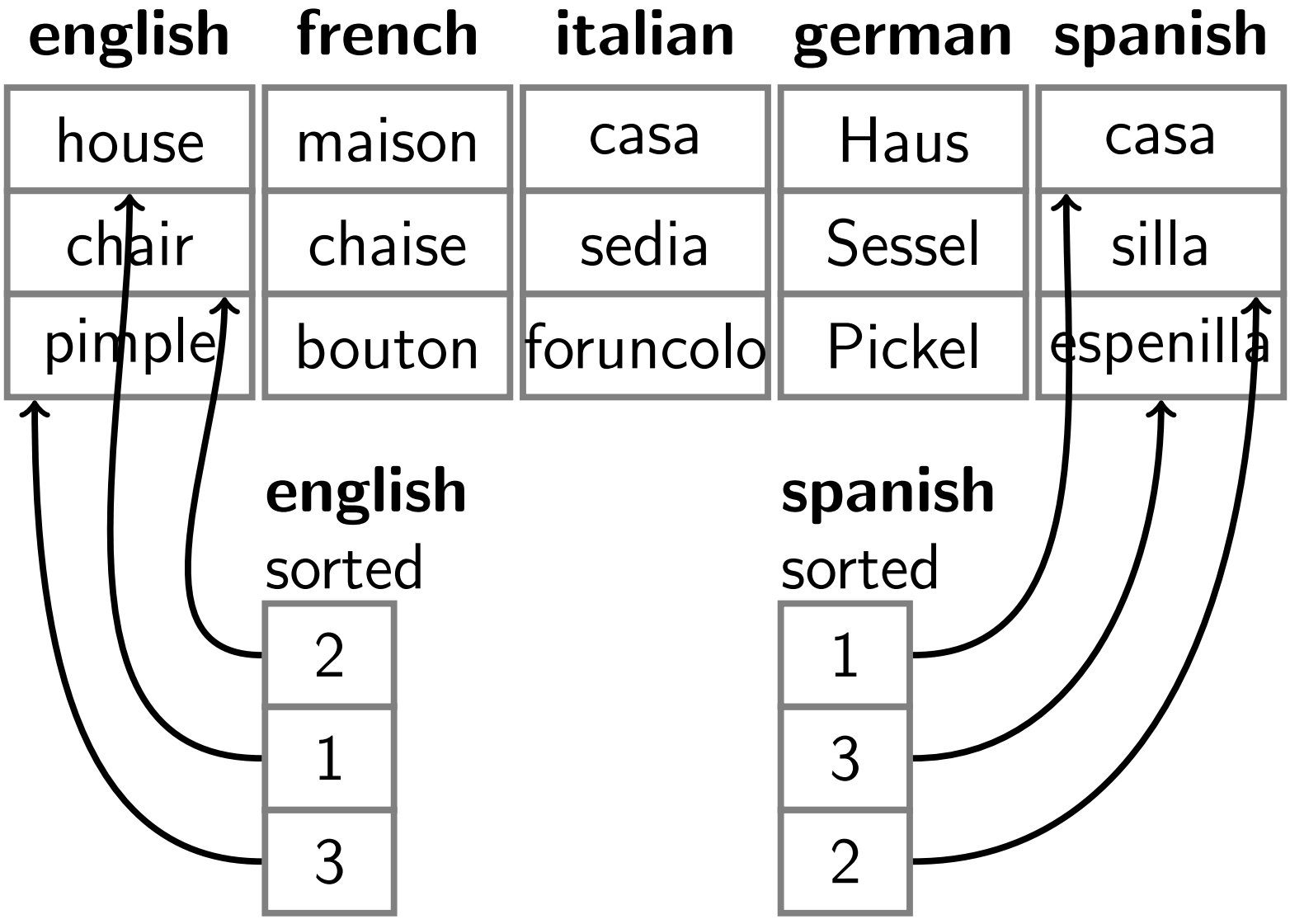
# Real-life Example



# Real-life Example



# Real-life Example



# Summary

# Summary

The runtime of binary search is $\Theta(\log n)$.

# Outline

1. What is the Master Theorem
2. Proof of Master Theorem

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$



$$T(n) = O(\log n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n^2)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n^{\log_2 3})$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$



$$T(n) = O(n \log n)$$

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases}$$

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \end{cases}$$

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ \end{cases}$$

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

# Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

# Master Theorem Example 1

$$T(n) = 4\,T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

# Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

$$b = 2$$

# Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 4$$

$$b = 2$$

$$d = 1$$

# Master Theorem Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$a = 4$$

$$b = 2$$

$$d = 1$$

Since $d < \log_b a$, $T(n) = O(n^{\log_b a}) = O(n^2)$

# Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

# Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

# Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

$$b = 2$$

# Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 3$$

$$b = 2$$

$$d = 1$$

# Master Theorem Example 2

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$a = 3$$

$$b = 2$$

$$d = 1$$

Since $d < \log_b a$,
$T(n) = O(n^{\log_b a}) = O(n^{\log_2 3})$

# Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

# Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

# Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

$$b = 2$$

# Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^1)$$

$$a = 2$$

$$b = 2$$

$$d = 1$$

# Master Theorem Example 3

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2$$

$$b = 2$$

$$d = 1$$

Since $d = \log_b a$,
$T(n) = O(n^d \log n) = O(n \log n)$

# Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

# Master Theorem Example 4

$$T(n) = 1\,T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

# Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

$$b = 2$$

# Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(n^0)$$

$$a = 1$$

$$b = 2$$

$$d = 0$$

# Master Theorem Example 4

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$a = 1$$

$$b = 2$$

$$d = 0$$

Since $d = \log_b a$, $T(n) = O(n^d \log n) = O(n^0 \log n) = O(\log n)$

# Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

# Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

# Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

# Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

$$d = 2$$

# Master Theorem Example 5

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$a = 2$$

$$b = 2$$

$$d = 2$$

Since $d > \log_b a$, $T(n) = O(n^d) = O(n^2)$

# Outline

1. What is the Master Theorem
2. Proof of Master Theorem

# Master Theorem

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

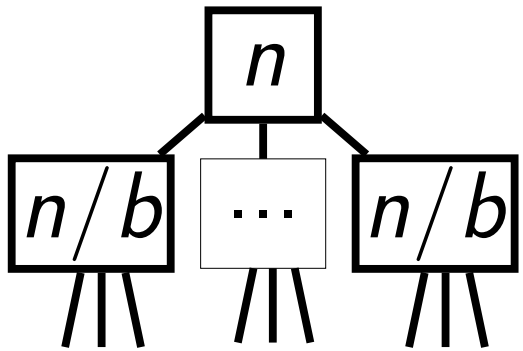
$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

level

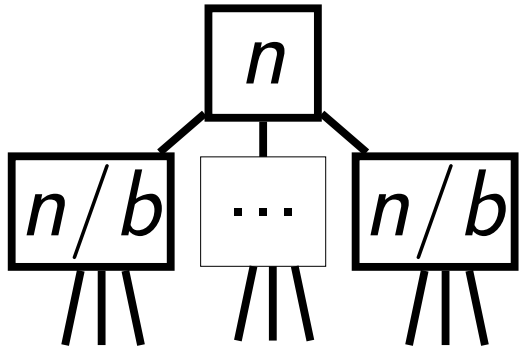


$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

level

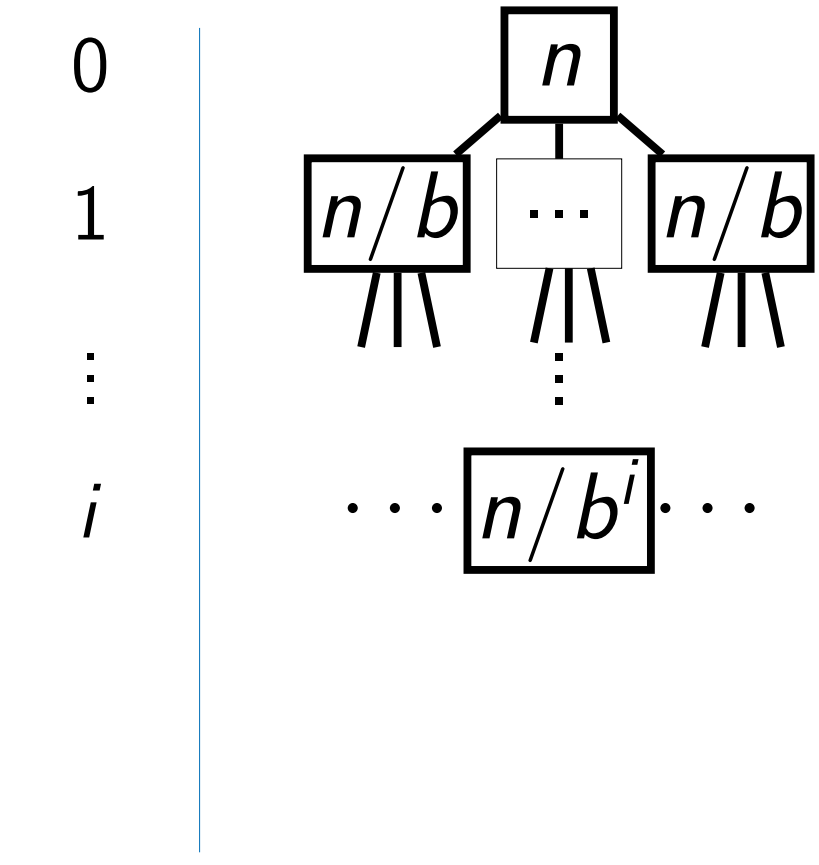
0

1

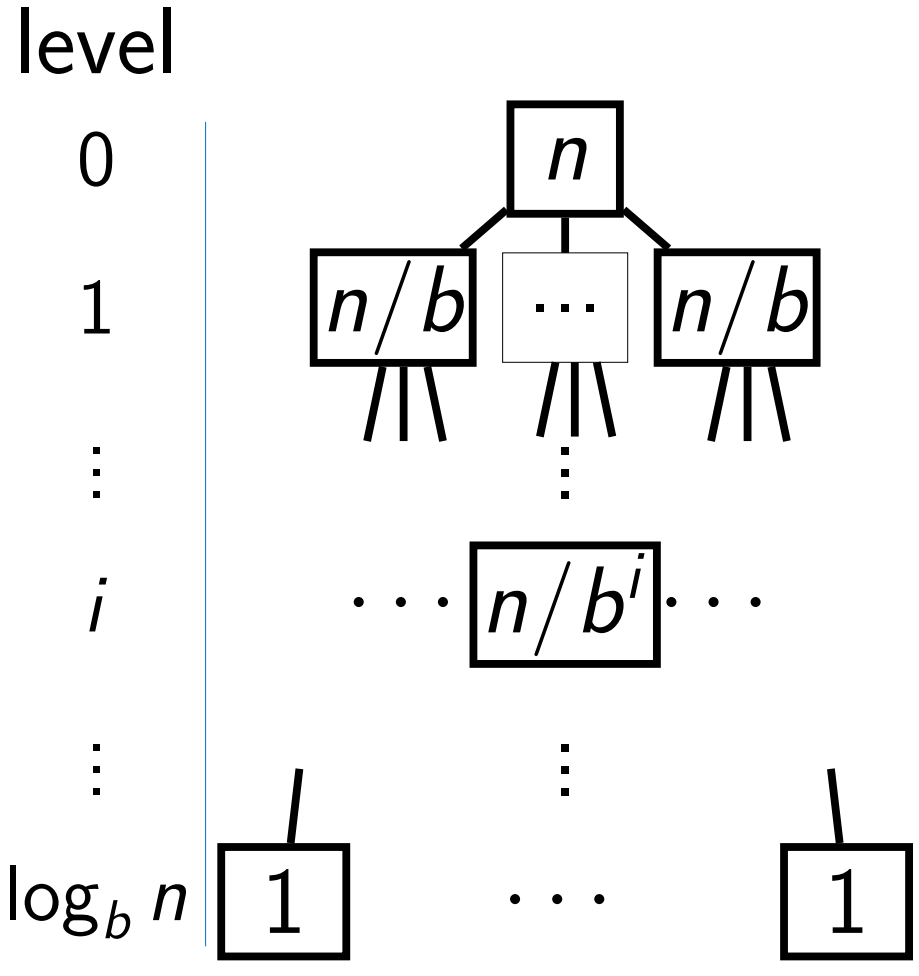


$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$

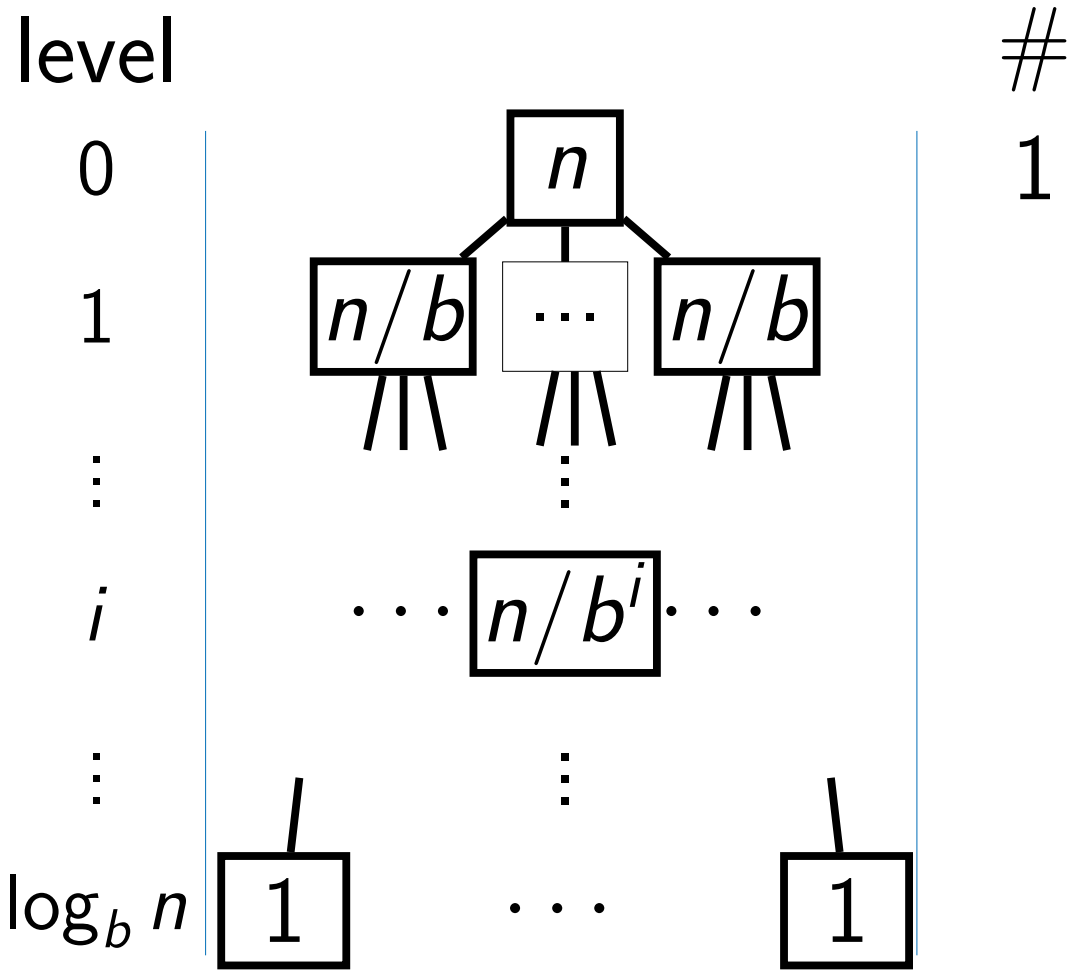
level



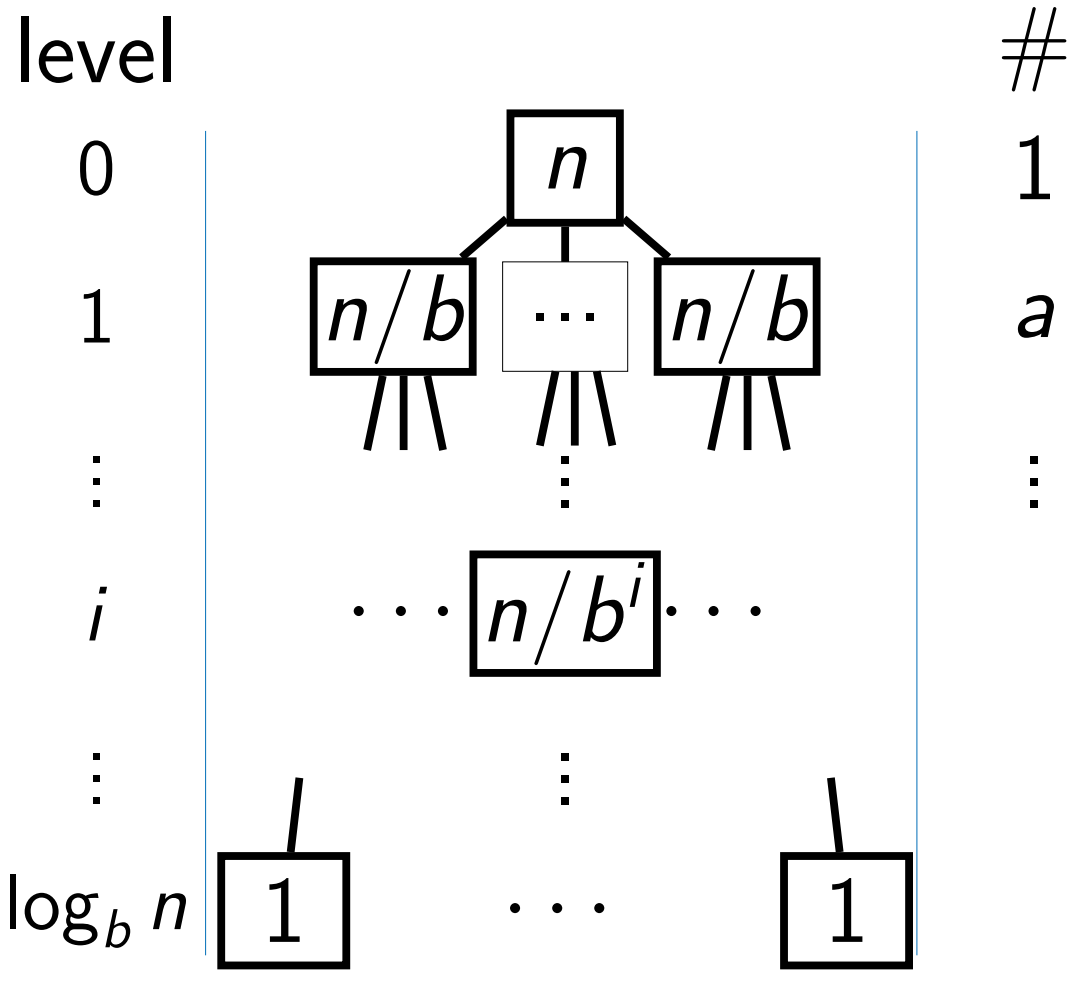
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



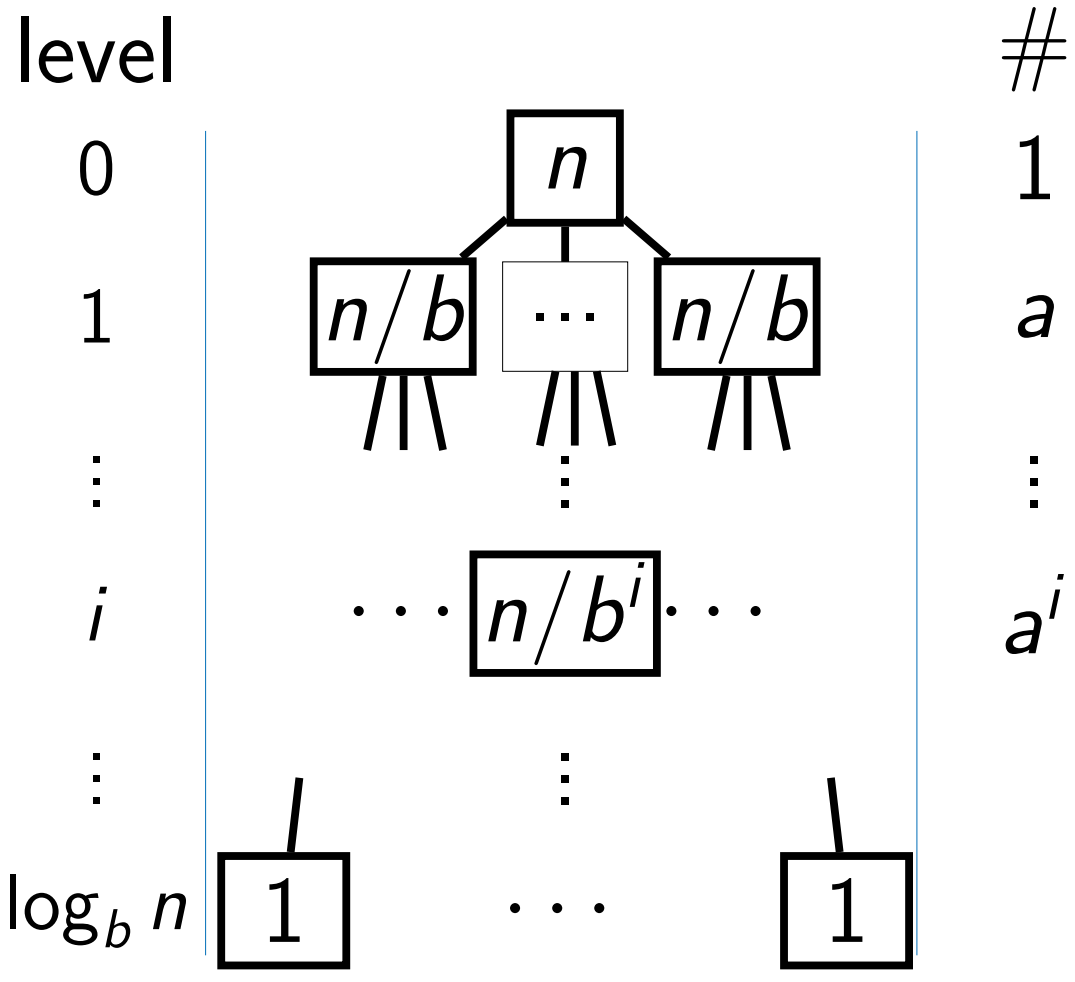
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



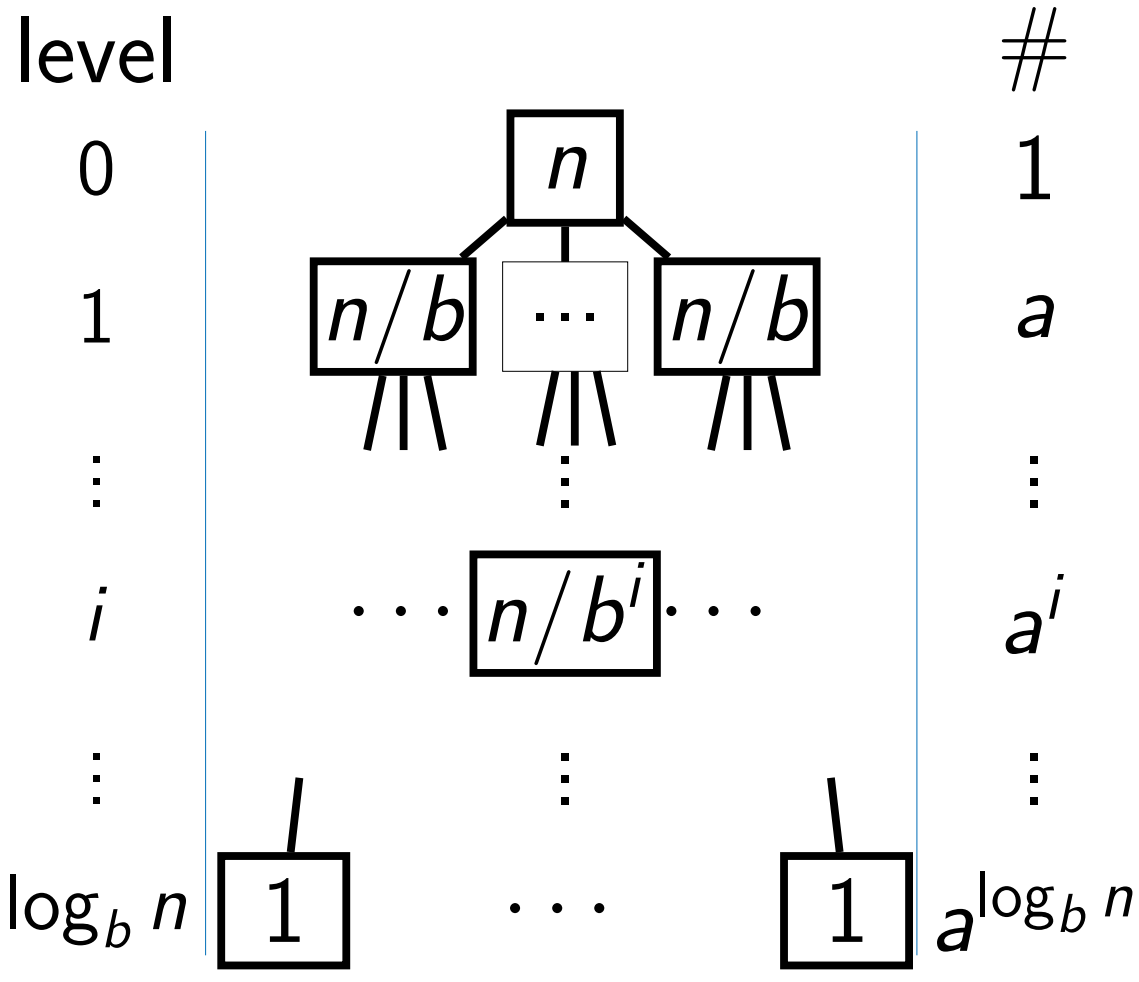
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



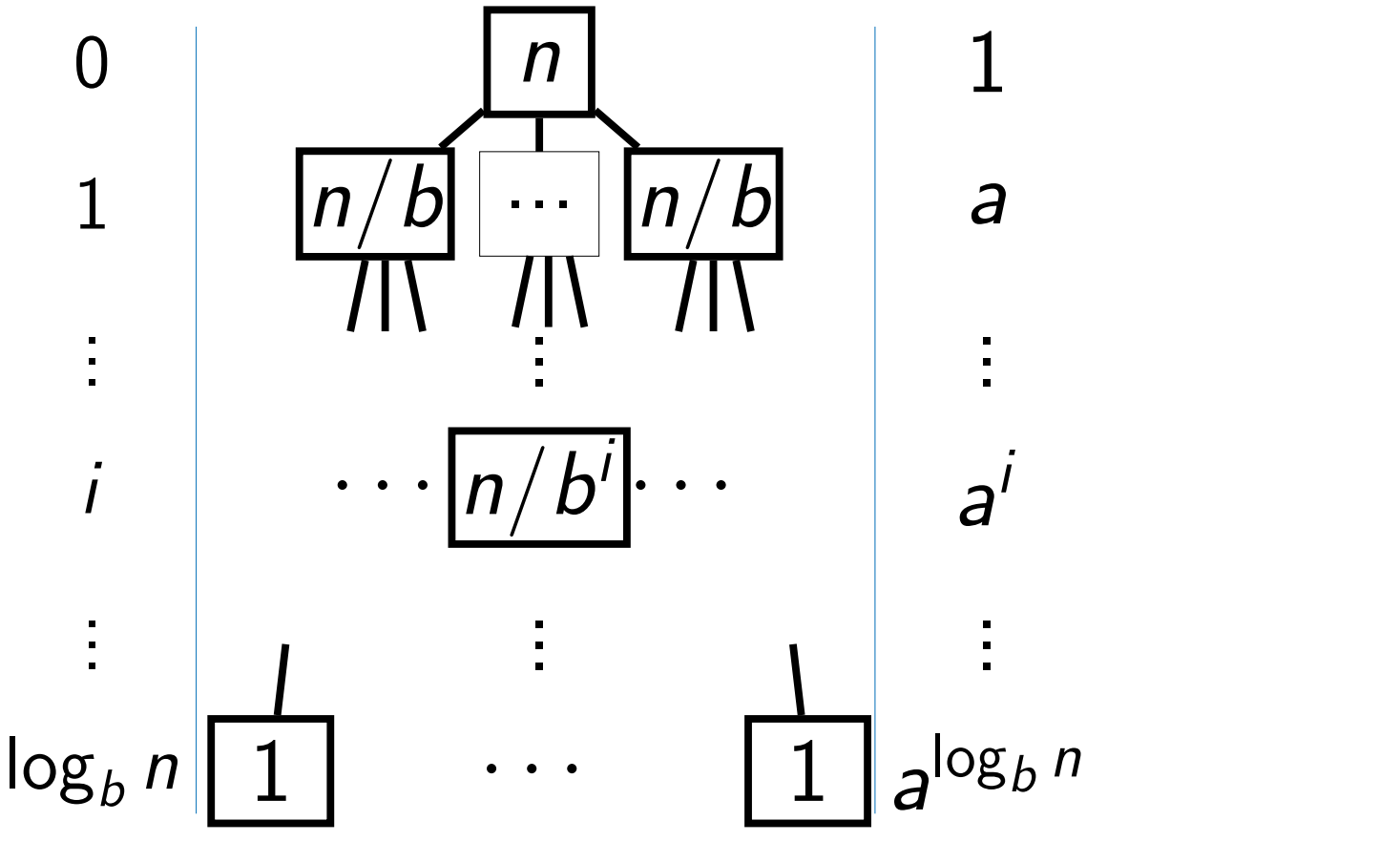
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



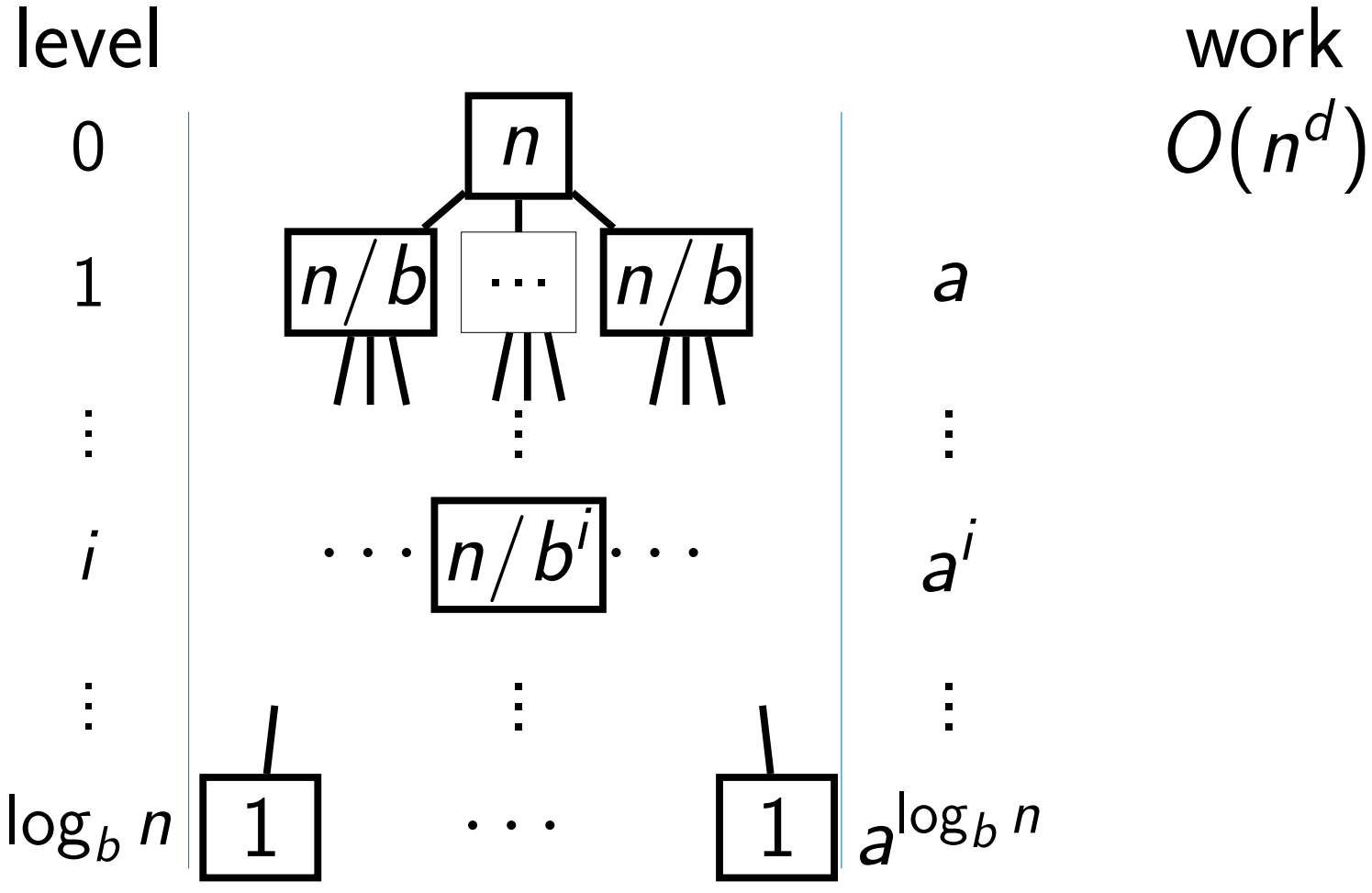
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



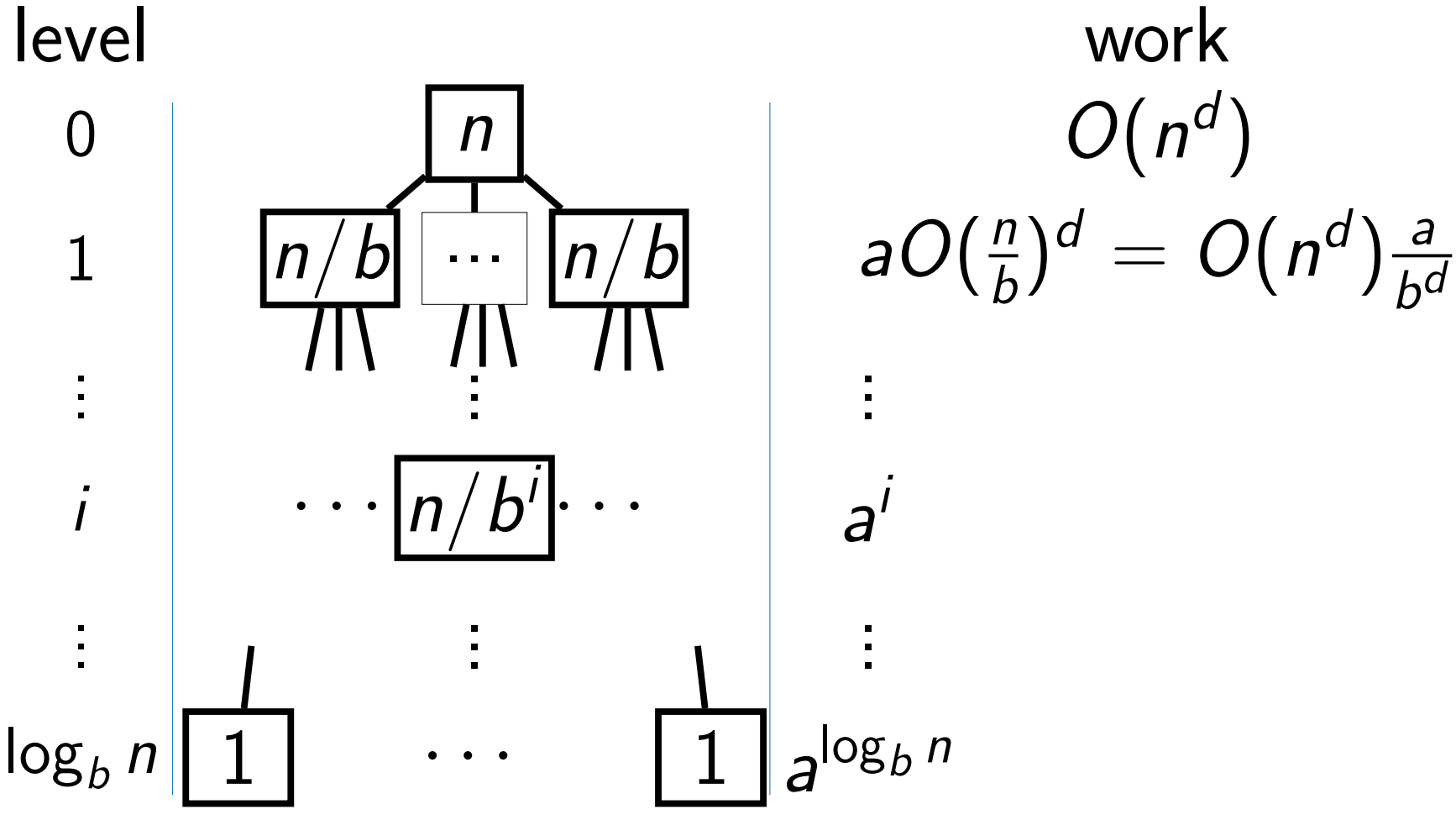
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



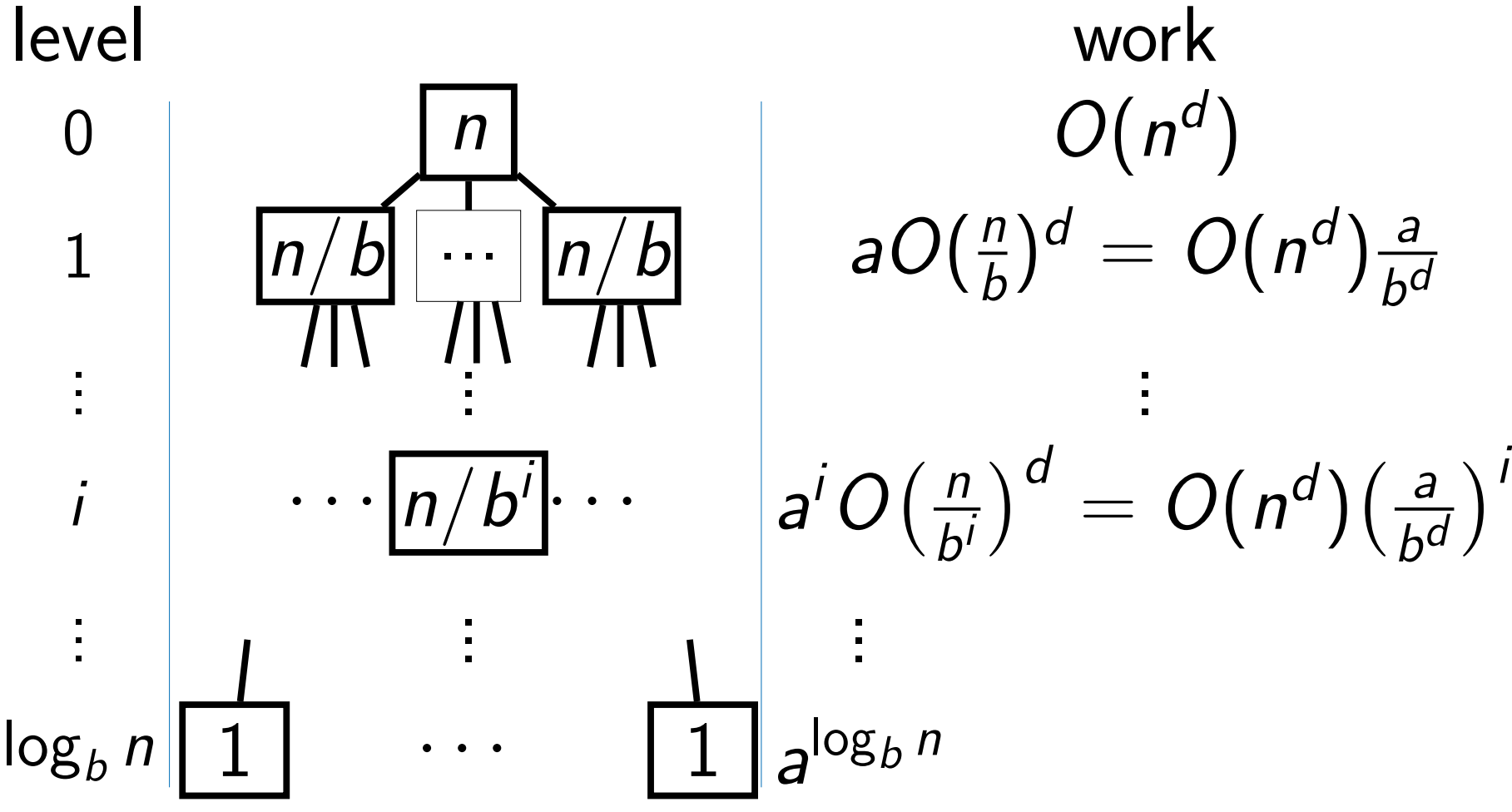
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



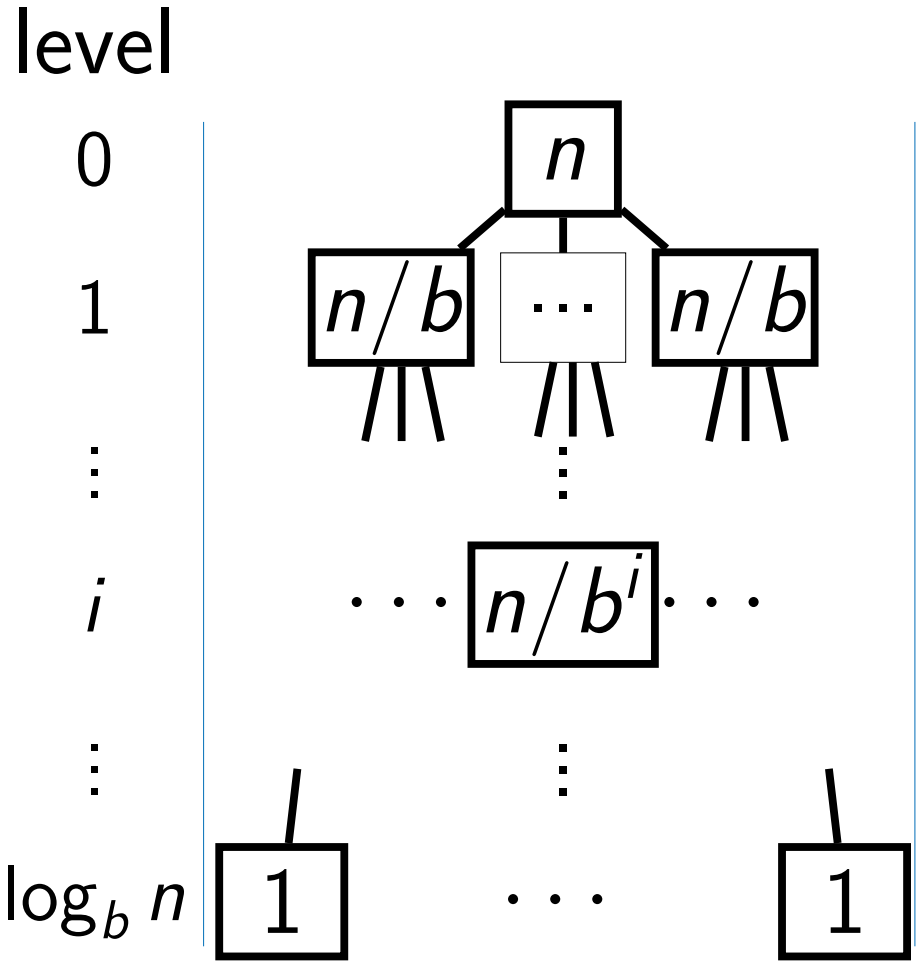
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



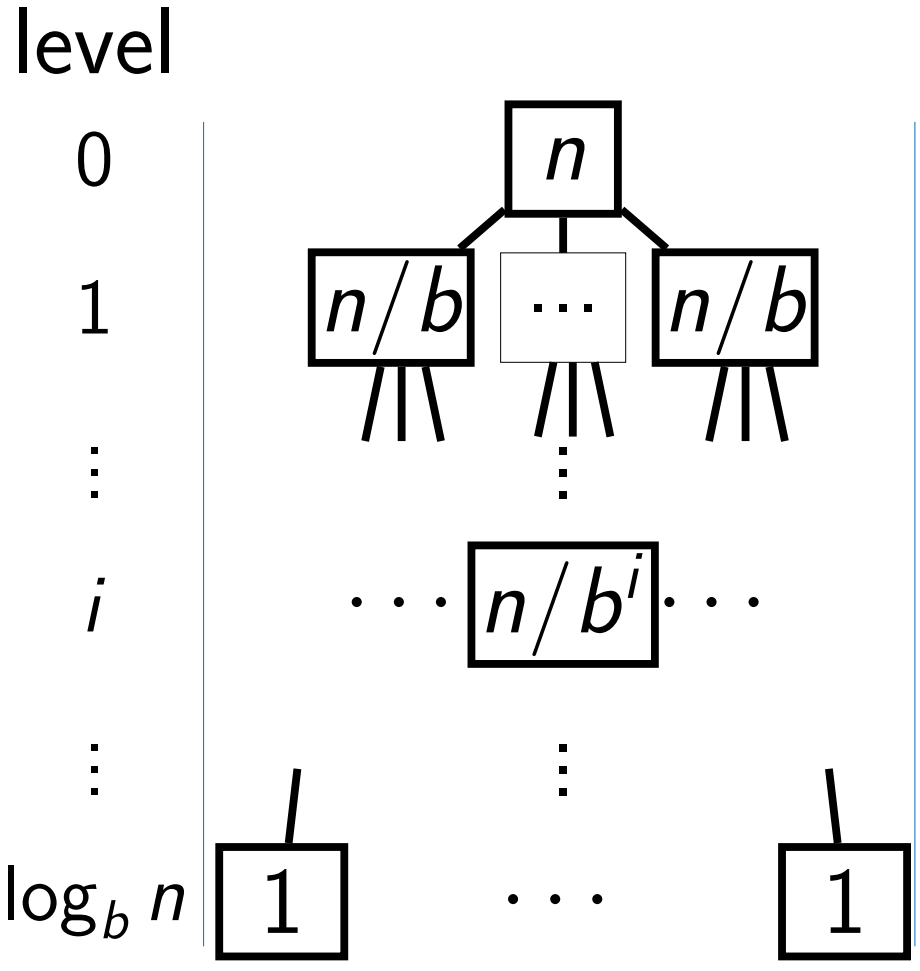
$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$$



# Geometric Series

For $r \neq 1$:

$$a + ar + ar^2 + ar^3 + \cdots + ar^{n-1}$$

# Geometric Series

For $r \neq 1$:

$$\begin{aligned} & a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\ =& a\frac{1 - r^n}{1 - r} \end{aligned}$$

# Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases}
\end{cases}
\end{aligned}
$$

# Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases} O(a) & \text{if } r < 1 \\ \end{cases}
\end{aligned}
$$

# Geometric Series

For $r \neq 1$:

$$
\begin{aligned}
& a + ar + ar^2 + ar^3 + \cdots + ar^{n-1} \\
=& a\frac{1 - r^n}{1 - r} \\
=& \begin{cases} O(a) & \text{if } r < 1 \\ O(ar^{n-1}) & \text{if } r > 1 \end{cases}
\end{aligned}
$$

## Case 1: $\frac{a}{b^d} < 1$ ($d > log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

## Case 1: $\frac{a}{b^d} < 1$ ($d > log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O(n^d)$$

# Case 2: $\frac{a}{b^d} = 1$ $(d = log_b a)$

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

# Case 2: $\frac{a}{b^d} = 1$ $(d = log_b a)$

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

# Case 2: $\frac{a}{b^d} = 1$ ($d = log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

$$= (1 + \log_b n) O(n^d)$$

# Case 2: $\frac{a}{b^d} = 1$ ($d = log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= \sum_{i=0}^{\log_b n} O(n^d)$$

$$= (1 + \log_b n) O(n^d)$$

$$= O(n^d \log n)$$

# Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

# Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

# Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

# Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

$$= O\left(O(n^d)\frac{n^{\log_b a}}{n^d}\right)$$

# Case 3: $\frac{a}{b^d} > 1$ ($d < log_b a$)

$$\sum_{i=0}^{\log_b n} O(n^d)\left(\frac{a}{b^d}\right)^i$$

$$= O\left(O(n^d)\left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

$$= O\left(O(n^d)\frac{a^{\log_b n}}{b^{d \log_b n}}\right)$$

$$= O\left(O(n^d)\frac{n^{\log_b a}}{n^d}\right)$$

$$= O(n^{\log_b a})$$

# Summary

Master theorem is a shortcut:

# Summary

Master theorem is a shortcut:

## Theorem

If $T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$